

**VEHICLE RENTAL AND
RESERVATION MANAGEMENT
SYSTEM FOR
‘SETHYANA RENT A CAR & CAB
SERVICE’**

by

M.V.G.R. THARAKA

April 2022



VEHICLE RENTAL AND RESERVATION MANAGEMENT SYSTEM FOR ‘SETHYANA RENT A CAR & CAB SERVICE’

M.V.G.R. THARAKA

Client: A.P. Chamal Wijerathna

Supervisor: K.R.A. Lalith Kahatapitiya

April 2022



**This dissertation is submitted in partial fulfilment of the requirement of the
Degree of Bachelor of Information Technology (External) of the
University of Colombo School of Computing**

Declaration

"I certify that this dissertation does not incorporate, without acknowledgement, any material previously submitted for a degree or diploma in any university and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text. I also hereby give consent for my dissertation, if accepted, to be made available for photocopying and for interlibrary loans, and for the title and abstract to be made available to outside organization.

Signature of Candidate:  Date: 28 / 04 / 2022

Name of Candidate: M.V.G.R. Tharaka

Countersigned by:

Signature of Supervisor(s)/Advisor(s):  Date: 28 / 04 / 2022

Name(s) of Supervisor(s)/Advisor(s): K.R.A. Lalith Kahatapitiya

Abstract

Sethyana Rent a Car & Cab Service is a reputable leading transport service provider in Avissawella area. The company provides a wide range of vehicles for rent, from economic to luxury, with options to self-drive or drive with a chauffeur. The company has been running its everyday business on a manual paper-based system. The manual approach is inefficient and time-consuming when making new reservations, registering new customers, and updating vehicle details. The proposed web-based system is developed to handle the company's primary business operations effectively, including making new reservations, allocating drivers and vehicles, and registering new customers and vehicles. The system payment module is designed to perform accurate calculations while making the payment process fast. Moreover, the system is facilitated to generate reports and notifications promptly, which is helpful in decision-making. The system is developed as a web application based on guidelines of iterative and incremental development approach, considering the ambiguity of some of the requirements and the ability to improve and enhance each developed version of the product through iterations. The system has adopted Model View Controller (MVC) architecture following techniques of object-oriented concepts. Java is used as the primary programming language, whereas the spring framework is used to manage security requirements, making the development and testing of the application fast. In addition, MySQL is used to create the application's database, and hibernate is used for object-relational mapping for Java. Apache tomcat is used as the webserver. Further, Unified Modeling Language (UML) was used to analyze and design the system. Integrating Vehicle Rental and Reservation Management System (VRMS) will improve the company's business functionalities while eliminating existing issues with the manual method. Accordingly, the company will be able to improve customer satisfaction by providing rapid and effective service to its customers while maximizing profits and revenue.

Acknowledgement

I owe many thanks to the people who supported me along the way in successfully completing this project. First of all, I would like to express my gratitude to the BIT coordinators and academic staff of the University of Colombo School of Computing (UCSC) for giving us, as students, a valuable opportunity to dive into the emerging IT industry with excellent knowledge and skill set.

I express my sincere gratitude to my project advisors, Mr. Lalith Kahatapitiya and Mr. Haritha Pramodha, for providing unstinted support and guidance to complete the project successfully.

I would like to give my humble appreciation to Mr. A.P. Chamal Wijerathna, the owner of Sethyana Rent a Car & Cab Service, for being very kind and supportive from the beginning of this project. His willingness to give his valuable time was a tremendous privileged.

I also would like to make this an opportunity to extend my heartfelt thanks to my family members and my dear friend for their encouragement and motivation towards the success of my academic life.

Table of Content

Declaration.....	iii
Abstract.....	iv
Acknowledgement	v
Table of Content.....	vi
List of Figures.....	xi
List of Tables	xiii
List of Acronyms	xiv
Chapter 1 – Introduction.....	1
1.1 Project Motivations	1
1.2 Project Objectives.....	2
1.3 Project Scope	3
1.4 Outline of Remaining Chapters	4
Chapter 2 –Analysis	6
2.1 Review of Similar Systems	6
2.1.1 HQ Car Rental Software	6
2.1.2 RentSyst.....	7
2.2 The Current Manual System.....	9
2.3 Identify and Gather System Requirements.....	11
2.3.1 Functional Requirements	11
2.3.2 Non-functional requirements	13
2.4 Selected Process Model	15
Chapter 3 – Design	16
3.1 Selected Designed Strategy	16
3.2 Alternative Solutions of Design	16

3.1.1 Standalone System	16
3.1.2 Open-source Software Components.	17
3.3 Object Oriented Design	17
3.4 UML Diagrams.....	17
3.4.1 Use Case Diagram for the Proposed System	17
3.4.2 Class Diagram of Proposed System.....	19
3.4.3 Entity Relationship Diagram of Proposed System	19
3.4.4 Sequence Diagram	22
3.4.5 Activity Diagram	23
3.5 User Interface	24
3.5.1 User Login.....	24
3.5.2 System Dashboard	25
3.5.3 System Table Design	26
3.5.4 System Form Design.....	27
3.5.5 Reports Generation	30
3.5.6 Invoice Design	31
Chapter 4 – Implementation	32
4.1 Implemented Environment	32
4.2 Tools and Technologies Used	33
4.2.1 Java Language.....	33
4.2.2 MySQL	33
4.2.3 Spring Framework	34
4.2.4 Hibernate.....	34
4.2.5 Apache Tomcat	34
4.2.6 Gradle.....	35
4.2.7 IntelliJ IDEA.....	35
4.2.8 MySQL Workbench.....	35

4.2.9 Visual Paradigm.....	35
4.3 Code and Module Structure.....	36
4.3.1 MVC Architecture	36
4.3.2 The Model Implementation	37
4.3.3 The Controller Implementation	39
4.3.4 The View Implementation	42
4.4 Reused Existing Codes and APIs	44
Chapter 5 – Evaluation.....	46
5.1 Testing Strategies	46
5.1.1 Unit Testing	46
5.1.2 Integrated Testing	46
5.1.3 System Testing.....	46
5.2 Test Plan	47
5.3 System Test Cases	48
5.3.1 Test Cases for Customer Module	48
5.3.2 Logging Related Test Cases	49
5.3.3 Test Cases for Package Module.....	50
5.3.4 Test Cases for Common Functions.....	51
5.4 User Evaluation	53
Chapter 6 – Conclusion	56
6.1 The Final Evaluation	56
6.2 Future Enhancement.....	57
6.3 Lessons Learnt.....	57
References	58
Appendix A – System Manual.....	61
A.1 Hardware requirements	61
A.2 Software requirements.....	61

A.3 Installation & Configuration.....	61
A.3.1 Install Java Runtime Environment.....	61
A.3.2 Installation of MySQL Server.....	63
Appendix B – Design Documentation.....	65
B.1 Entity Relationship (ER) Diagrams	65
B.1.1 ER Diagram of Vehicle Management Module.....	65
B.1.2 ER Diagram of Package Management Module.....	66
B.1.3 ER Diagram of Chauffeur-Drive Reservation Management Module	67
B.1.4 ER Diagram of Customer Management Module	68
B.2 Use Case Diagrams & Narratives	69
B.2.1 Use case diagram for Manager’s Activities	69
B.2.2 Use narrative diagram for Manager’s Activities.....	69
B.3 Sequence Diagram	70
B.3.1 Sequence Diagram for Login to the System	70
Appendix C – User Manual.....	71
C.1 Main Window Related Components.....	71
C.1.1 Sidebar.....	72
C.1.2 Nav Bar	72
C.2 Table Related Components.....	74
C.2.1 Table Header	74
C.2.2 Pagination.....	74
C.3 User Login	75
C.4 Creating a New Chauffeur-Drive Reservation.....	76
C.4.1 Time & Location Tab.....	76
C.4.2 Package Tab	77
C.4.3 Vehicle Tab	77
C.4.4 Driver Tab	78

C.4.4 Customer Tab	79
C.4.4 Confirm Tab	79
C.4.5 Printing Payment Receipt.....	81
Appendix D – Management Reports	82
D.1 Chauffeur-Drive Rental Reports.....	82
D.2 Revenue Reports.....	83
Appendix E – Test Results.....	84
E.1 Test Results for Customer Module	84
E.2 Test Results for Common Test Cases	86
Appendix F – Client Certificate.....	88
Glossary.....	89
Index.....	90

List of Figures

Figure 2.1: Vehicle Page of HQ Car Rental Software.	7
Figure 2.2: Reservation Page of RentSyst Software.	8
Figure 2.3: Use Case Diagram for the Current Manual System.	10
Figure 2.4: Development Flow of Iterative and Incremental Process Model [5].	15
Figure 3.1: Use Case Diagram of Proposed System.	18
Figure 3.2: Class Diagram of the Proposed System.	20
Figure 3. 3: ER Diagram of the Proposed System.	21
Figure 3. 4: Sequence Diagram for Making New Reservation.	22
Figure 3. 5: Activity Diagram for Making New Reservation.	23
Figure 3. 6: Interface of User Login.	24
Figure 3. 7: Interface of System Dashboard.	25
Figure 3. 8: Table Interface of Customer Management.	26
Figure 3. 9: Table Interface of Vehicle Management.	27
Figure 3. 10: Interface of Vehicle Registration.	28
Figure 3. 11: Interface of Chauffeur-Drive reservation form: Time & Location Tab. .	28
Figure 3. 12: Interface of Chauffeur-Drive reservation form: Confirm Tab.	29
Figure 3. 13: Interface of Report Generation Module.	30
Figure 3. 14: Generated Rental Agreement.	31
Figure 4.1: Diagram of MVC architecture [19].	36
Figure 5.1: User Evaluation Form-1	54
Figure 5.2: User Evaluation Form-2	55
Figure A.1: Java Runtime Environment Installation-Welcome Page.	62
Figure A.2: Java Runtime Environment Installation-Destination Folder.	62
Figure A.3: Java Runtime Environment Installation-Finish.	63
Figure A.4: MySQL Server Installation-Setup Type.	63
Figure A.5: MySQL Server Installation-Select Products.	64
Figure B.1: ER Diagram of Vehicle Management Module.	65
Figure B.2: ER Diagram of Package Management Module.	66
Figure B.3: ER Diagram of Chauffeur-Drive Reservation Management Module.	67
Figure B.4: ER Diagram of Customer Management Module.	68
Figure B.5: Use Case Diagram of Manager's Activities.	69

Figure B.6: Sequence Diagram of the System Login.....	70
Figure C.1: Dashboard of the System.	71
Figure C.2: Sidebar of the System. Figure C.3: Mini Sidebar of the System.	72
Figure C.4: Nav Bar of the System.	72
Figure C.5: Breadcrumb of the system.	72
Figure C.6: Quick Access of the System.	73
Figure C.7: Notifications of the System.....	73
Figure C.8: Logged User Profile.....	73
Figure C.9: Table Related Components.	74
Figure C.10: Table Header	74
Figure C.11: Table Pagination.	74
Figure C.12: User Login of the System.	75
Figure C.13: Chauffeur-Drive Reservation-Time & Location Tab.	76
Figure C.14: Chauffeur-Drive Reservation-Package Tab.....	77
Figure C.15: Chauffeur-Drive Reservation-Vehicle Tab.....	78
Figure C.16: Chauffeur-Drive Reservation-Driver Tab.....	78
Figure C.17: Chauffeur-Drive Reservation-Customer Tab.....	79
Figure C.18: Chauffeur-Drive Reservation-Confirm Tab.....	80
Figure C.19: Generated Customer Payment Receipt.	81
Figure F.1: Client Certificate.	88

List of Tables

Table 4.1: Hardware & Software Requirements	32
Table 5.1: Test Cases for Customer Module.....	49
Table 5.2: Login Related Test Cases.	49
Table 5.3: Test Cases for Package Module.....	51
Table 5.4: Test Cases for Common Functions.....	53
Table A.1: System Hardware Requirements.....	61
Table A.2: System Software Requirements.	61
Table B.1: User Case Narrative for Manager's Activities.....	69
Table E.1: Test Results for Customer Module.....	86
Table E.2: Test Results of Common Test Cases.	87

List of Acronyms

API – Application Programming Interface
BIT – Bachelor of Information Technology
CASE – Computer Aided Software Engineering
CRM – Customer Relationship Management
CRUD – Create Read Update Delete
CSS – Cascading Style Sheets
DBMS – Database Management System
ERD – Entity Relationship Diagram
GPS – Global Positioning System
GUI – Graphical User Interface
HQL – Hibernate Query Language
HTTP - HyperText Transfer Protocol
IDE – Integrated Development Environment
IT – Information Technology
JPA - Java Persistence API
JPQL - Java Persistence Query Language
MVC – Model-View-Controller
OOD – Object Oriented Design
OOP – Object Oriented Programming
ORM – Object Relational Mapping
POJO – Plain Old Java Object
REST – REpresentational State Transfer
SDLC – System Development Life Cycle
SQL – Structured Query Language
TDD – Test Driven Development
UCSC – University of Colombo School of Computing
UI – User Interface
UML – Unified Modeling Language
VRMS – Vehicle Rental & Reservation Management System

Chapter 1 – Introduction

Transportation is a crucial aspect of society nowadays concerning the busy life we tend to pursue. Whether commuting to work, traveling, or attending an event, people who do not own personal vehicles and those who do both favor renting vehicles for their convenience.

Identifying the rising demand, Sethyana Rent a Car and Cab Service was established in 2018 in Avissawella under the registered number WE 5641. The company provides a wide range of vehicles for rent under very competitive rates with options to self-drive or drive with a chauffeur. More than 15 vehicles, including cars, passenger vans, SUVs, and mini-busses, are functioning daily. Meanwhile, the company employs 12 personnel, including drivers and staff members.

1.1 Project Motivations

The company undertakes all the reservations over the phone and records details on paper manually. Searching for existing clients and retrieving information about upcoming trips are very inefficient. In addition to using the current paper-based system, the company has encountered the problems mentioned below.

- Registering a new customer, a new vehicle, and making a new reservation take too much time of the customer.
- Filling paper-based forms have no data validation mechanism, which has led to entering inaccurate and duplicated records.
- Daily available vehicle fleet and drivers are not updated on time, therefore leading to conflicts when allocating them to new reservations.
- Timely tasks such as renewing revenue licenses, vehicle insurances, and driver licenses have been missed and often have been performed after an extended due date.

- Recording driver commissions for each completed ride has been inconsistent, thereby has arisen conflicts while making driver payments.
- There is no organized and systematic way to view upcoming trips and rentals; hence allocating vehicles has been complex.
- There is no proper mechanism to manage vehicle maintenance, damage and repair details, which have led to breakdowns during rides.
- Daily and monthly reports take too much time to generate due to a lack of a systematic approach.

1.2 Project Objectives

The main objective of the proposed system is to overcome current problems, enhance and improve the company's daily business activities, and manage its business processes efficiently. Thereby company will be able to provide competitive and effective service for their customers. The primary objectives are as follows.

- Significantly Reduce the time it takes to register a new customer and vehicle and make new reservations.
- Automatically update vehicle fleet and drivers on time to allocate them efficiently for new reservations.
- Implement enhanced filtering and searching capabilities in modules, making retrieval of records and documents faster and more relevant.
- With the notification module, timely tasks, such as vehicle revenue license, insurance, and driver's license are notified on time.
- Payment modules ensure the payment process is reliable and calculations are accurate.
- Manage maintenance and damage details of vehicles to prevent breakdowns during rides.

- Implement an effective report module, so that company can generate detailed reports promptly, which will aid with decision making.

1.3 Project Scope

The Scope of the project has been determined by the client's requirements and the devoted time. The scope for the proposed vehicle reservation management system is mentioned hereafter.

- All the company's rentals run based on various packages, and the customer will be charged for additional consumed time and kilometers at the end of the journey.
- The system has facilitated registering new users and giving them relevant system privileges to perform actions.
- The system allows users to register new customers, drivers, and vehicles and update their details when required.
- When a customer calls, the system allows users to enter customer requirements, immediately find a suitable vehicle, allocate a driver and create a new reservation. If the customer has previously violated any terms and conditions of the company and is being backlisted, the system would not allow making a new reservation for that customer.
- When the driver logs into the system, he can see all the reservations allocated to him for the day. At the end of a trip, the driver will be shown the cost of the trip alongside other trip details, which can be printed and given to the customer. Further, the driver will be notified when a new reservation is allocated for him.
- When a user makes a self-drive reservation, the system will check for the validity of his driving license first. And when the customer visits the office to pick up the vehicle, the system will display the total cost for the trip and will be asked to pay the advance. Subsequently, as the customer returns the vehicle, the system will re-calculate the price for actual rented days of the customer and charge a fine, if any.

- The system is facilitated to manage driver commission and fuel payments for relevant trips.
- The system can generate detailed reports, including timely income and vehicle usage reports, and also facilitates retrieving any details regarding reservations for a given period.
- Most of the modules are integrated with searching, filtering, and printing capabilities.
- Top-level management can create rental packages for different vehicle models and update them when required.
- The system is facilitated to check the availability of vehicles, drivers, and packages for a given period.
- With the notification module, events such as vehicle maintenance, driving license expiration and revenue license expirations will be notified on time.

1.4 Outline of Remaining Chapters

This section summarizes the contents of the remaining chapters of this dissertation, which includes an additional five chapters, references, six appendices, a glossary, and an index.

Chapter 2: Analysis

This chapter reviews similar systems with references and explains the current manual system with a high-level use case diagram. Then we discuss various requirements gathering techniques and the proposed system's captured functional and non-function requirements. It also discusses the selected process model to develop the software system and the reasons for choosing the particular one.

Chapter 3: Design

This chapter discusses the selected design strategy and other alternative solutions of design. It also briefly describes the object-oriented design and its essential concepts.

The chapter has provided several UML diagrams covering important design aspects of the system to give a better understanding of the architectural design of the system. Finally, under the sub-section of user interface design, it explains all the considerations goes into the careful design of the main components of system user interfaces.

Chapter 4: Implementation

This chapter incorporates the implementation of the software system. It shows the implemented environment and covers all the tools and technologies used to develop the system and the justification for choosing each technology in developing the system. Next, we discuss the code and module structure of the application. Here, we introduce MVC architecture and its implementation in the application while providing actual code examples and explanations. The last sub-section is dedicated to mentioning the reused codes and APIs.

Chapter 5: Evaluation

This chapter discusses how testing is done in the proposed system. It contains various testing strategies as well as the test plan. System test cases are provided for several main modules. Furthermore, it discusses how user evaluations are done and include some user evaluation forms filled by the company users.

Chapter 6: Conclusion

This chapter is the final chapter of the dissertation. it includes the final evaluation of the project and future enhancements that can be made to the developed software system. The chapter is ended with discussing the lessons learned by taking this great endeavor.

Appendices

The appendices contain further information that could be useful to interested readers of the dissertation. There are six appendices in the dissertation, which are named A to F.

Chapter 2 –Analysis

Requirement analysis is one of the most critical stages in the software development process; hence all the other phases proceed after it depending on the requirements specified in this phase. The system requirements analysis aims to obtain a thorough and detailed understanding of the business process and break it down into discrete requirements, which are then reviewed and agreed upon. Requirement analysis for the proposed project is done by studying the current manual process, reviewing similar vehicle rental systems, initiating interviews and questioners with employees, and reviewing various company documents.

2.1 Review of Similar Systems

Studying similar systems helps to identify not only the core functional requirements in a similar domain but also allows to get ideas about how non-functional requirements have been integrated with these systems. It also helps to discover different user interface designs and reveals bad design choices and implementations. Following are the similar systems that are studied.

2.1.1 HQ Car Rental Software

HQ Car Rental Software is a popular choice among many vehicle rental systems. It covers all the basic functionalities one would expect from a vehicle reservation management system, including fleet management, rentals and reservation management, customer payments management, and vehicle maintenance management. It comes with additional features like real-time vehicle tracking and barcode scanning. The system is integrated with a customer relationship management (CRM) module allowing managers to view customer profiles and their rental history. The system is provided with robust filtering and searching capabilities for its critical modules, enabling the user to retrieve only relevant information. The system is facilitated to generate a number of reports such as revenue reports, expenses reports, and monthly overview reports just to mention a few.

Although HQ Car Rental Software provides a variety of functionalities, it comes with a greater cost, \$990 per year for the basic package [1] which manages up to 25 vehicles.

Additionally, this software solution does not meet the client's requirement of package management; the software is not facilitated to create of packages based on distance but only on time. Moreover, it is not possible to separately create packages for self-drive and chauffeur-drive rentals; instead, the driver charge is independent of package cost and treated as an additional service that can be selected while making a reservation. Form validation is somewhat low-level, leaving the user to enter incorrect data into the system. The following Figure 2.1 shows the vehicle page of HQ Car Rental Software.

Vehicle Key	Vin	License Plate	Status	Vehicle Type	Current Renter	Available Date	Make	Vehicle Model	Vehicle Class	Current Location	Odo
0001	5YYVQRTMECQHFSFVE	1	Rental	Car	#1 Brandt Balistreri		Opel	Antara 3.2 V6	Economic Manual	Office	158.1
0002	SXEMGSAZRUHIXVOBG	11	Dirty	Car	Not Assigned		Opel	Meriva 1.8 Elegance Automatic	Economic Manual	Office	197.8
0003	55L2MEQVUTP0IA70P	21	Dirty	Car	Not Assigned		Opel	Vectra 2.2 Automatic	Economic Manual	Office	12.51
0004	XPLUPAUWHK5FEY0YO	31	Rental	Car	#4 Ariel Abbott		Ford	Anglia	Economic Manual	Office	73.68
0005	WNODQFCYCTIVPRVY6	41	Dirty	Car	Not Assigned		Peugeot	406 2.0 ST Break	Economic Manual	Office	150.8

Figure 2.1: Vehicle Page of HQ Car Rental Software.

2.1.2 RentSyst

RentSyst is a cloud-based software application that helps streamline car rental businesses' workflow. It handles all the basic operations, including fleet and reservation management, vehicle and maintenance management, and customer management. Further, the system has provided the facility to track vehicle indicators such as mileage, fuel level, and location. [2]

Visual pleasing and thoughtfully designed user interface can be seen throughout the system, which is also mobile-responsive. Creating a new reservation is guided through steps that help increase the process's clarity. Many important business analytics and

charts are accessible to the system user on the dashboard as well as in generatable reports. Moreover, the daily plan and timeline tabs show upcoming rentals.

The main drawback of the RentSyst software system is that it barely meets the client's requirement of package management since the software is mainly focused on self-drive rentals. The following Figure 2.2 shows the reservation page of RentSyst software.

Id	Info	Status	Pickup	Return	Vehicle	Customer	Amount	Actions
205	Created: 24-01-2022 18:20 Demo Manager: Iala	reserved	Jan 1 2022 03:00 RENT A BOAT	Jan 12 2022 02:00 RENT A BOAT	MITSUBISHI ASX Ekonomik NKVT92	LK LEONIDAS KAKARIARIS Individual Clients	Total: 12263,00 € Due balance: 12263,00 €	[Icons]
204	Created: 24-01-2022 16:43 Demo Manager: Iala	rental	Jan 3 2022 03:00 RENT A BOAT	Jan 11 2022 05:00 RENT A BOAT	CITROEN BERLINGO Ekonomik	AB ARGJEND BAJRAKTARI Individual Clients	Total: 0,00 € Due balance: 0,00 €	[Icons]
203	Created: 18-01-2022 15:33 Demo Manager: Iala	done	Feb 2 2022 21:01 RENT A BOAT	Feb 15 2022 03:29 RENT A BOAT	AUDI A1 Ekonomik 12334	SD SH DO Individual Clients	Total: 220,10 € Due balance: 0,00 €	[Icons]

Figure 2.2: Reservation Page of RentSyst Software.

Considering the cost of purchasing these systems, the level of customization they provide, and the inability to meet some of the client's requirements, it was decided that developing a new software system is the right direction. However, analyzing similar systems that exist in the market has helped in many ways in designing the proposed system.

2.2 The Current Manual System

Managing reservations is one of the main business processes of the Sethyana company among the others. There are two main reservation types, self-drive reservations and chauffeur-drive reservations. Self-drive reservations are the ones that permit the customer to drive the vehicle by himself. By choosing the self-drive rental, customers can rent the vehicle up to 30 days after paying a refundable deposit. Whereas, Chauffeur-drive reservations allow customers to rent a vehicle with a driver for up to 5 days. When a customer calls the office, the receptionist will start to take down the specified requirements on a paper sheet, make sure the vehicle and drivers are available, and inform the customer.

All the company's vehicles are rented based on packages, and the manager creates those packages. The manager will also calculate monthly revenue reports. Furthermore, the Receptionist is responsible for updating the daily available vehicle fleet and drivers and processing and collecting customer payments. All those activities had been done manually on paper, which had consumed a lot of time and effort and also pruned to errors due to a lack of systematic approach. The following Figure 2.3 shows the top-level use case diagram for the existing manual system of Sethyana company.

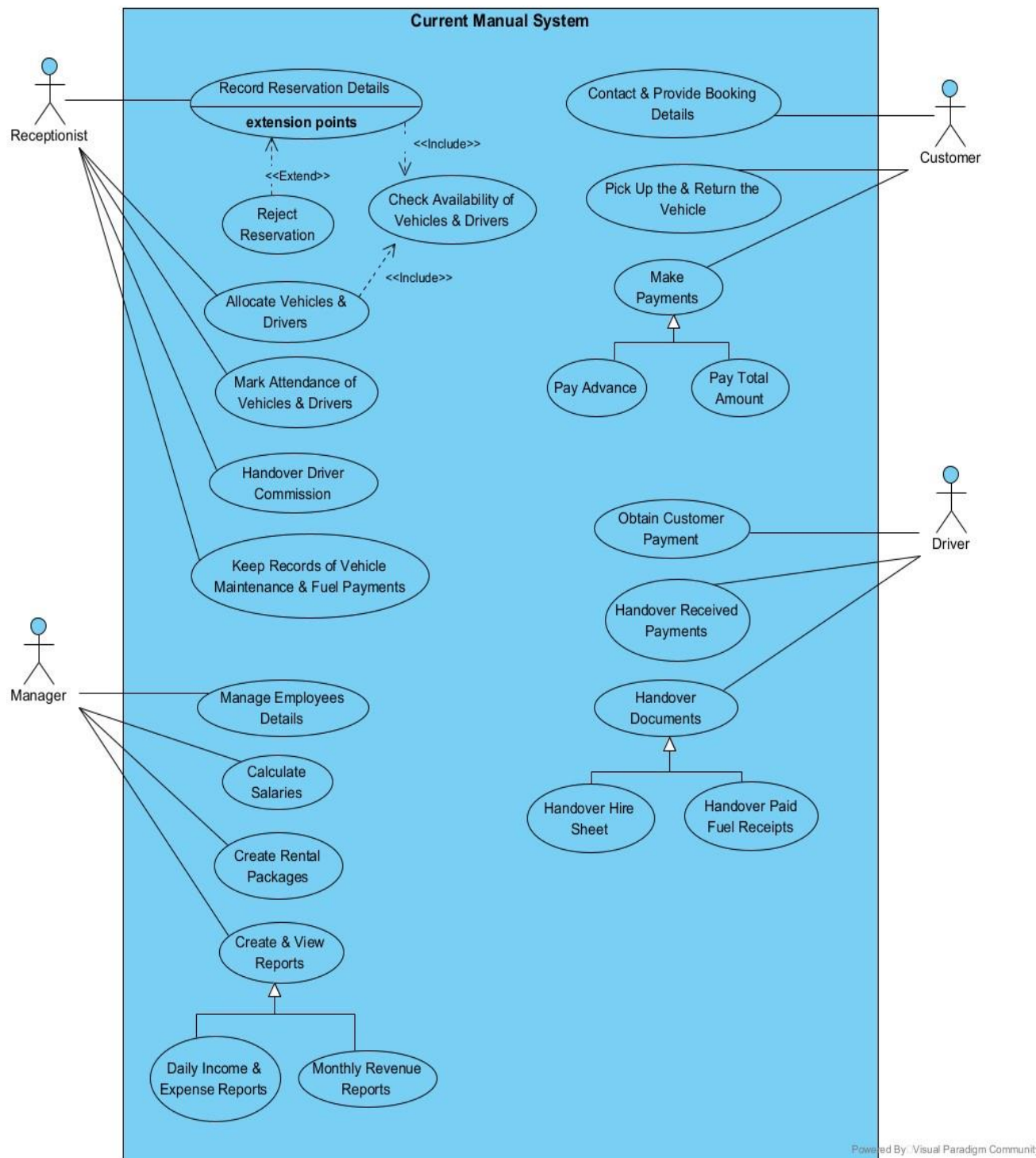


Figure 2.3: Use Case Diagram for the Current Manual System.

2.3 Identify and Gather System Requirements

Requirement gathering, also known as requirement elicitation, is a crucial part of the requirement analysis phase. Inadequate information and misunderstanding of the client's expectations may lead to the failure of the developing software project. This project's requirements were gathered through interviews, on-site observations, and analysis documentation.

Several interviews were conducted with the owner, which helped to gain client expectations of the system to be developed. Also, interviews with the manager helped to understand the company's administrative process and workflow. Being able to get immediate feedback is the most beneficial aspect of interviewing over the other requirement-gathering techniques. However, arranging the time for interviews over busy schedules can be tricky.

Document Analysis is much more effective in getting into details in a shorter period. Vehicle reservation forms and agreements, company terms and conditions, payment invoices, and package details were among the documents analyzed.

Observation also helped in understanding the practicality of the business domain and the flow of day-to-day operations. Observation with administrative permission aid in understanding the roles of the company and their duties and responsibilities as well.

2.3.1 Functional Requirements

Functional requirements are the services that the software should offer. It describes a software system or its component. A function is simply input to the software system, its behavior, and output. Calculations, data manipulations, business processes, or any user interactions can be considered functional requirements [3].

Functional requirements for the proposed system are mentioned hereafter.

- Manage system users and privileges

System users can be created only by the manager, and relevant privileges can be assigned to the users based on their role in the company. Registered users can log into the system securely by validating their usernames and passwords.

- Manage vehicle details

Vehicles are added to the system through a detailed vehicle registration form. Those details can be edited and updated as necessary.

- Manage package details

With the package module, various rental packages can be created for vehicle models by the manager and updated as needed.

- Manage Customer details

System users can add new customers and edit their details. The customer table shows all the customers in the database and allows users to filter and search by the name or NIC.

- Manage vehicle reservations

The vehicle reservation module is facilitated to enter the customer booking details into the system, search for available packages, and allocate vehicles and drivers based on their attendance. The reservation table shows all the reservations created and can quickly search any record by typing reservation id.

- Manage customer payments

Through this module, advance payments and total rental amounts can be calculated and printed to the customer.

- Manage fuel expenses

Fuel receipts paid to the filling stations by the driver are obtained and paid back to the driver by the office, and all the other fuel payments will be recorded here.

- Manage driver commissions

Driver commissions are paid after a driver has completed his monthly target.

- Manage attendance of vehicles and drivers

Users can mark the daily attendance of vehicles and drivers here.

- Check availability of packages, vehicles, and drivers

The system is facilitated to check the availability of packages, vehicles, and drivers at one place for a given period.

- Generate reports

Important reports such as daily income reports, expense reports, and monthly revenue reports can be generated by this module.

- Generate notifications

The system will be facilitated to generate important notifications such as vehicle revenue license and insurance expirations, vehicle maintenance notifications, and driver license expirations. Also, system-generated emails and text messages will be sent to the customer on confirmed reservation and driver arrival.

- Driver Portal

Drivers can log into their accounts on mobile or tablet devices by providing their user names and passwords. All the upcoming rentals can be seen in the main window. At the end of the journey, the total amount of the trip will be displayed and printed to the customer.

2.3.2 Non-functional requirements

The quality attribute of a software system is specified by its non-functional requirements, such as responsiveness, usability, security, and portability. In fact, they are critical to satisfying user needs and the success of the software system [4]. Non-functional requirements for the proposed system can be described as follows.

- Security

As security is concerned, it is one of the most critical non-functional requirements of a software system. The system is facilitated to create strong user passwords using alphanumeric combinations. Passwords are stored in the database only after a password is encrypted. The system would be locked for a user for a certain period if he tried to enter incorrect passwords consecutively more than five times. Using such strategies, unauthorized access to the system will be prevented. System privileges can be assigned to system users only by the manager so that the user can only perform authorized activities within the system.

- Reliability

Data input to the system is validated on the front-end as well as back-end using various techniques such as Regex and by implementing constraints on the database. By maintaining the accuracy of the information, Reliability is automatically maintained. Further, by taking regular data backups, the system can easily be restored in an emergency.

- Performance

The system is developed to work in maximum performance under minimum hardware and software requirements. The system is facilitated to run smoothly without crashing under a heavy load. Since this is a web application, it's important to reduce the consumption of users' data. Get requests have been optimized to bring only the limited data necessary. Only the minified files have been imported to the system, and the system is well bundled to reduce the number of files. Overall, these techniques will improve the system performance as well as the user experience.

- Usability

The system has been designed to minimize the learning curve. In fact, users can easily determine what a feature is and what it can do. Further, system interfaces have been designed by following core design principles such as color principles and principles of visual hierarchy. The sidebar and the quick access panel have been implemented for easy navigation options.

- Portability

Since the developed system is a web-based application, it can run on any operating system in which any modern browser is installed, whether it is windows, mac OS, Linux, or Android, given that the system is well tested on Google Chrome and Mozilla Firefox web browsers.

2.4 Selected Process Model

The iterative and incremental software development process model has been used to develop the proposed system. This process model is an iterative based software development approach that involves developing a software system by gradually increasing and improving its features through multiple iterations until the entire product is completed. Every iteration incorporates some level of requirements analysis, design, implementation, and testing. Planning is the first step in the iterative and incremental software development process, which continues through cycles of development that involve ongoing user feedback. At the end of each cycle, an incremental software product with added feature set will be delivered to the client.

Most of the requirements of the proposed system are known up front but anticipated to evolve over time. The selected process model welcomes and accommodates changes in requirements, given that the system can continue to be evolved through each iteration. Having a limited schedule for the project, requirements needed to be prioritized. It can easily be achieved by the iterative process model since it allows to focus on a specific feature set on each iteration. Moreover, the fact that the client is being able to provide feedback on each product increment and validate the developing system will result in delivering a successful software system that will be accepted by the client for which it was designed and avoid surprises at the end of development. The following Figure 2.4 shows the development flow of the iterative and incremental process model.

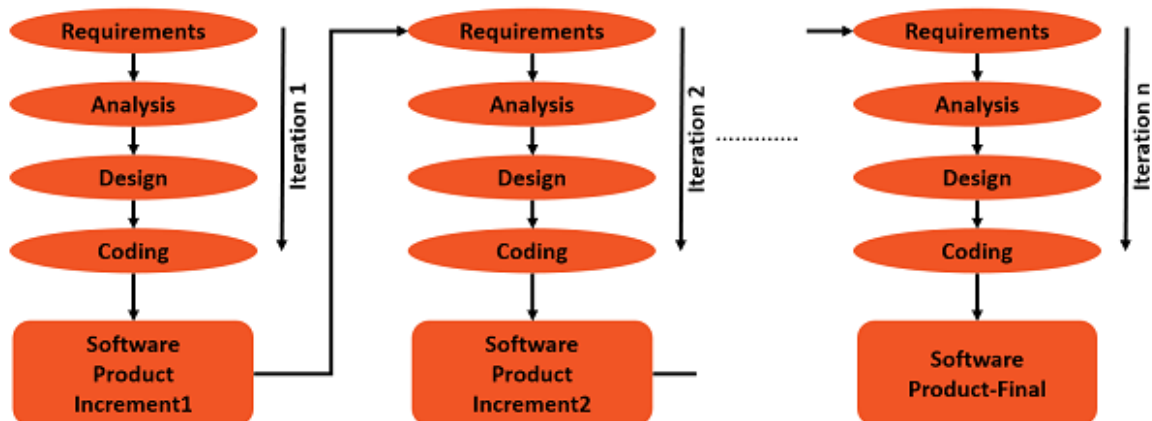


Figure 2.4: Development Flow of Iterative and Incremental Process Model [5].

Chapter 3 – Design

The system design phase is one of the critical stages performed after the requirement analysis. Requirements gathered in the analysis phase are translated into a blueprint to construct the system. Later this blueprint will be elaborated into a detailed system design specification. This chapter focuses on various design strategies and the selected ones and also includes multiple UML diagrams, which help to understand the design of system components from different perspectives. Also, it discusses the approach used to design user interfaces of some major modules of the system.

3.1 Selected Designed Strategy

It was decided to develop a web-based system from scratch, considering the advantages of a web-based application, including ease of use and familiarity, low cost for the software and hardware required for running the system, and being platform-independent. Also, adding new features and maintaining the system can be more manageable than other strategies. Further, being able to access the system from any location over the internet is a competitive advantage, especially considering that the company is going to open several branches in nearby cities in coming-up years as they scale up their business.

3.2 Alternative Solutions of Design

Several alternative design strategies can be identified for the proposed system. Among them are developing the system as a standalone application, using open-source components to build the system or building from scratch.

3.1.1 Standalone System

A standalone system is a software program that runs locally on the installed device. Majority of the standalone systems are platform-dependent. The risk of a security breach is considerably low since network connectivity is not required for the system to function. However, deployment and maintenance of these systems are somewhat time-consuming, as every terminal must be maintained individually. The usage of these systems is limited to one physical location. Considering those limitations and

drawbacks this design strategy was not entertained for developing the proposed software system.

3.1.2 Open-source Software Components.

Open-source software is any software that provides source code that can be modified and shared by anyone. Integrating these software components may assist in the rapid development and early delivery of the system. Although, one may still need to accept the terms and conditions defined in its license. Also, these systems can introduce new vulnerabilities and lack quality standards. Nevertheless, finding open-source software components that meet the client's requirements and the end goal was difficult.

3.3 Object Oriented Design

Using object-oriented design (OOD), the conceptual model is further refined into an object-oriented model. In the object-oriented design, the system is viewed as a collection of objects, and each object handles its state data. Object-oriented programming (OOP) is later used in the development phase to support the object-oriented design. Object oriented programming is a concept that works on the principles of abstraction, encapsulation, inheritance, and polymorphism. Additionally, unified modeling language has been used in designing the system since it supports object-oriented design.

3.4 UML Diagrams

Unified modeling language is a standardized modeling language that enables to specify, visualize, construct and document of the artifacts of software systems through sets of diagrams [6]. In the design phase, these diagrams were handy when communicating and validating the system's architectural design.

3.4.1 Use Case Diagram for the Proposed System

Use case diagrams help to identify the system's functionality and requirements using actors and use cases. Use cases describe services, tasks, or functions that a system is required to perform, whereas actors are the external entities that interact with the system. Use cases are one of the core concepts of UML modeling [7]. The use case diagram for the proposed system is depicted in Figure 3.1

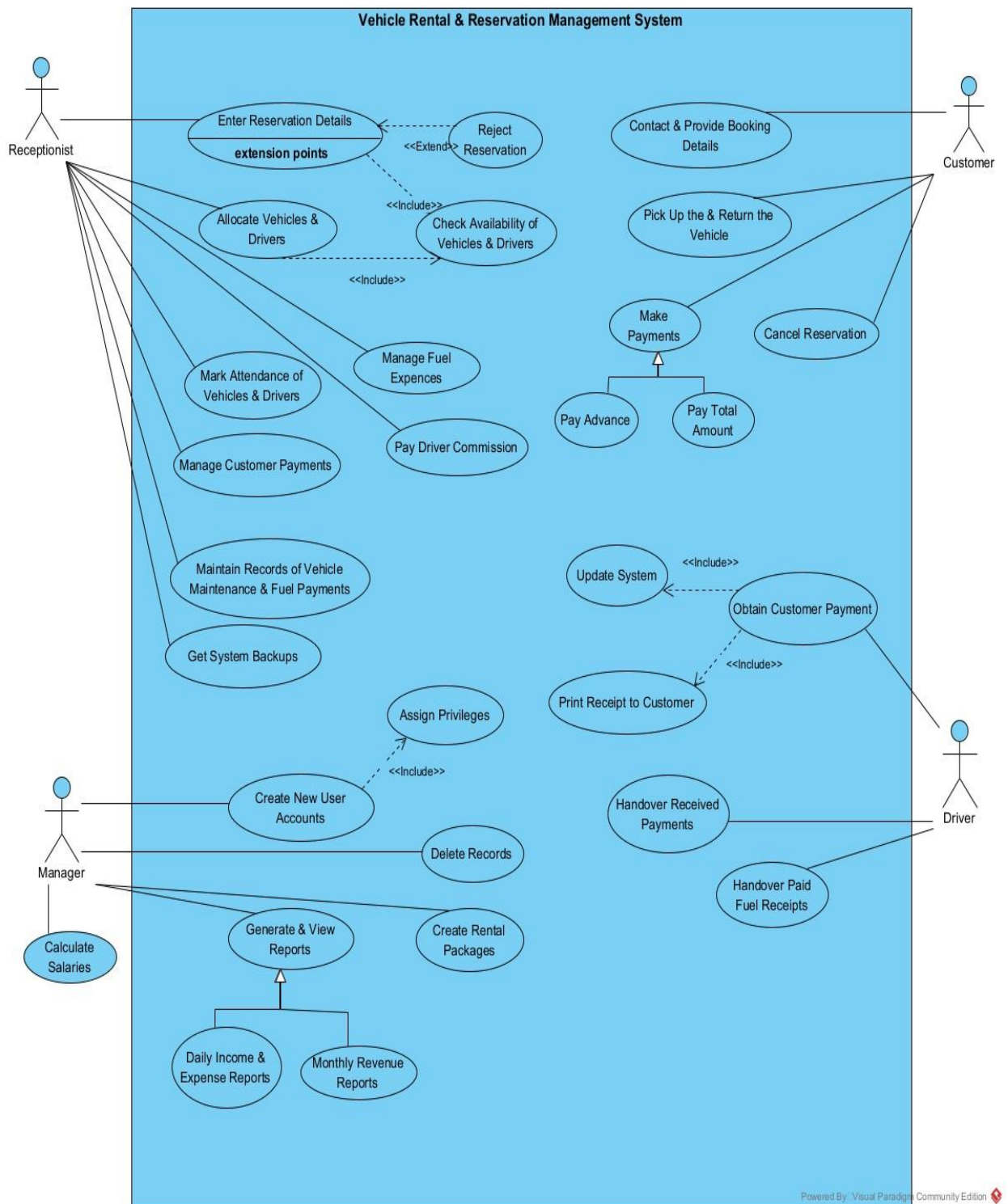


Figure 3.1: Use Case Diagram of Proposed System.

3.4.2 Class Diagram of Proposed System

A Class diagram is a static diagram that provides a static view of a software system by displaying classes, attributes, operations, and their connections to one another. This diagram includes attributes and operations of classes and also the constraints imposed on the system. The class diagram has been very helpful in developing the system since it can be mapped directly with object-oriented languages [8]. The class diagram for the proposed system can be seen in Figure 3.2.

3.4.3 Entity Relationship Diagram of Proposed System

Database design is an essential part of software development. Utilizing ER diagrams in database engineering guarantees a high-quality database design. Entity Relationship Diagrams, or ERDs for short, show the relationships between entity sets that are stored in the database. It also provides a means for communication of the logical structure of the database. ER diagrams are designed based on three fundamental concepts: entities, attributes, and relationships [9]. ER diagram for the proposed system can be seen in Figure 3.3.

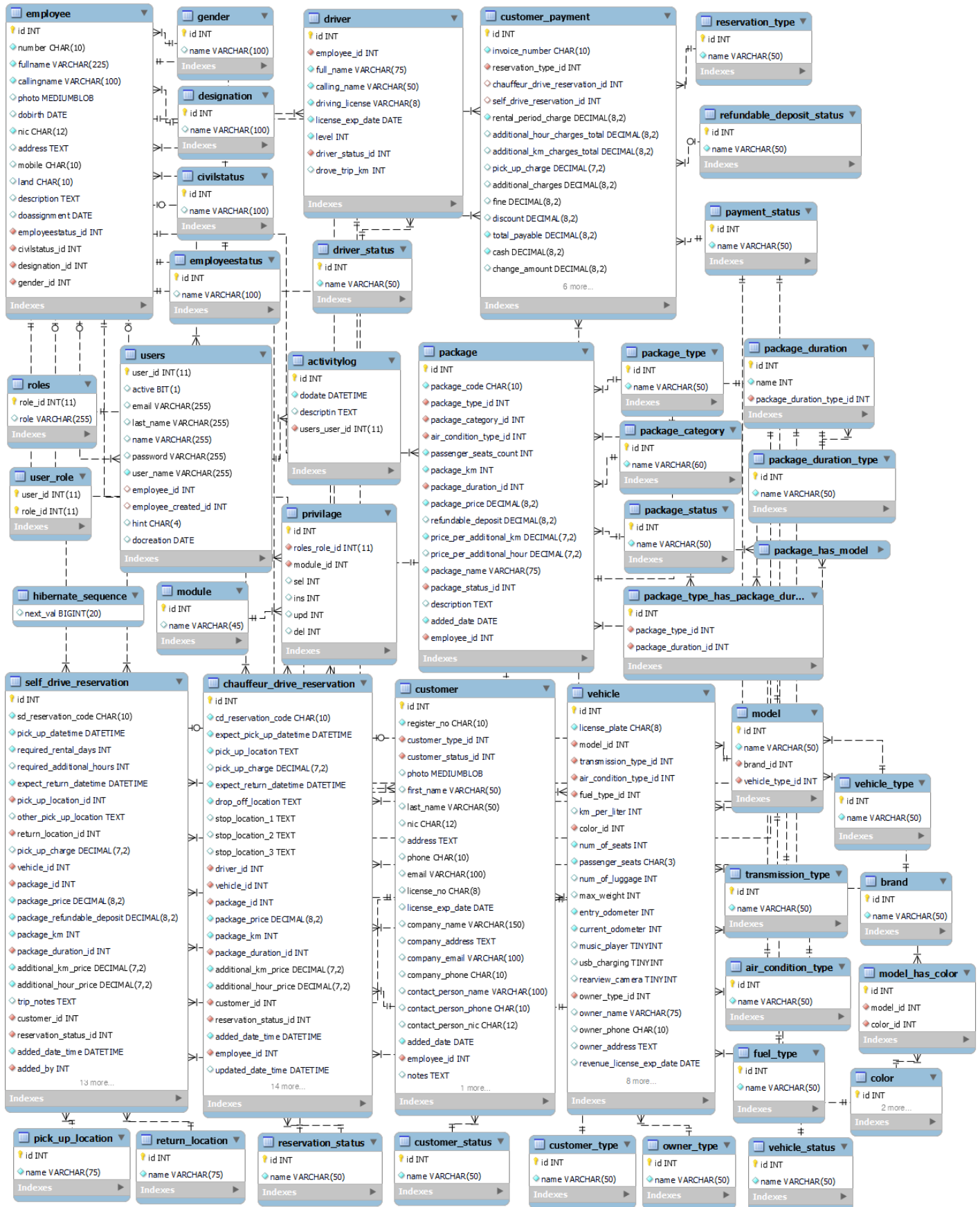


Figure 3. 3: ER Diagram of the Proposed System.

3.4.4 Sequence Diagram

A sequence diagram is an event diagram that depicts the interactions between objects in sequential order. In other words, it represents the flow of messages in the system. Software developers broadly use these diagrams to document and communicate the business requirements of new and existing software systems. The following Figure 3.4 shows the sequence diagram for making a new reservation.

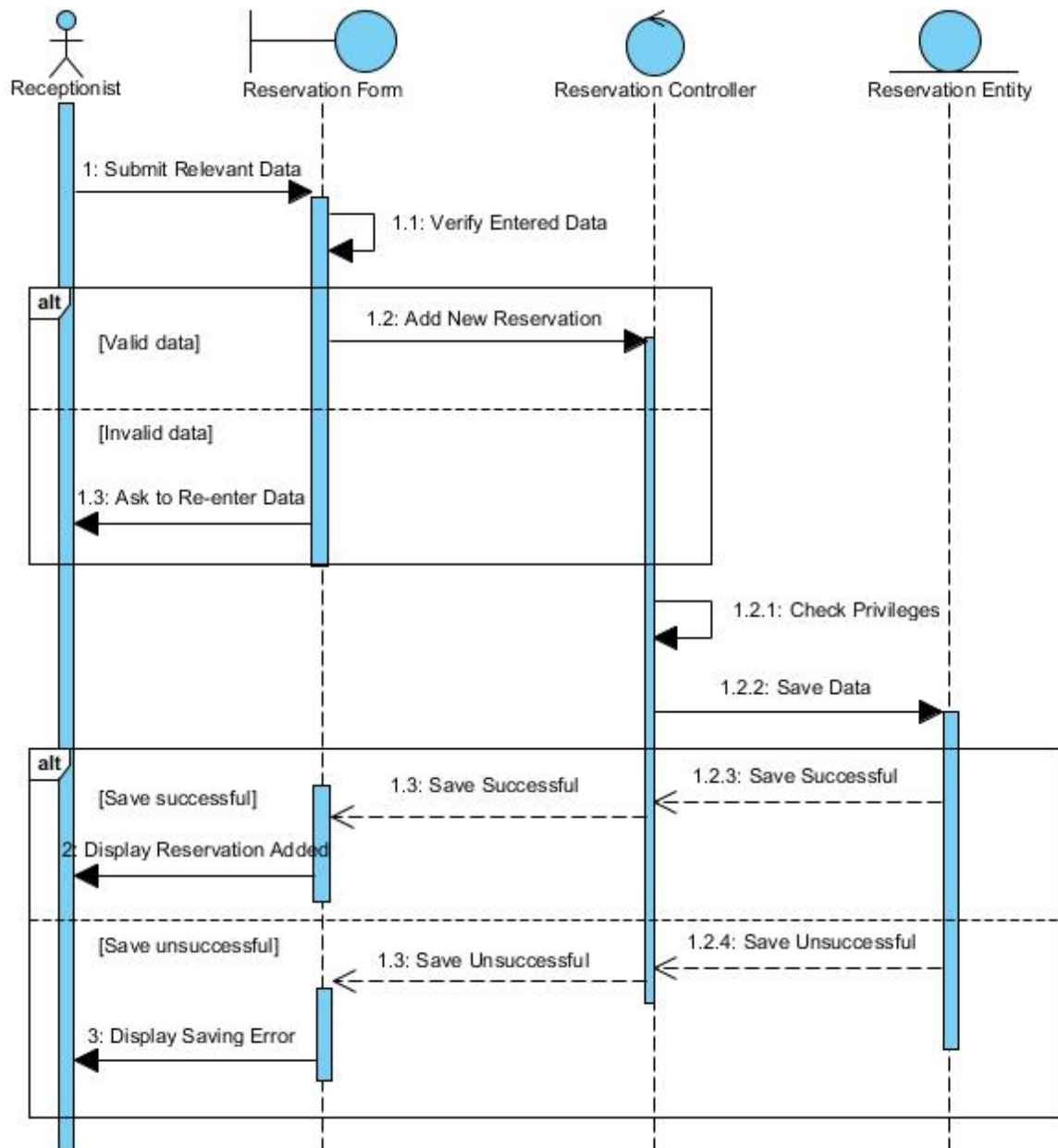


Figure 3. 4: Sequence Diagram for Making New Reservation.

3.4.5 Activity Diagram

Activity diagrams are important in describing the dynamic aspects of the system. In general terms, it is a flowchart representing the flow between activities of a system, and activities can be described as operations of a system. In order to capture the dynamic behavior of the proposed system, several activity diagrams were created for critical scenarios of the business domain. The following Figure 3.5 shows the activity diagram for making a new reservation.

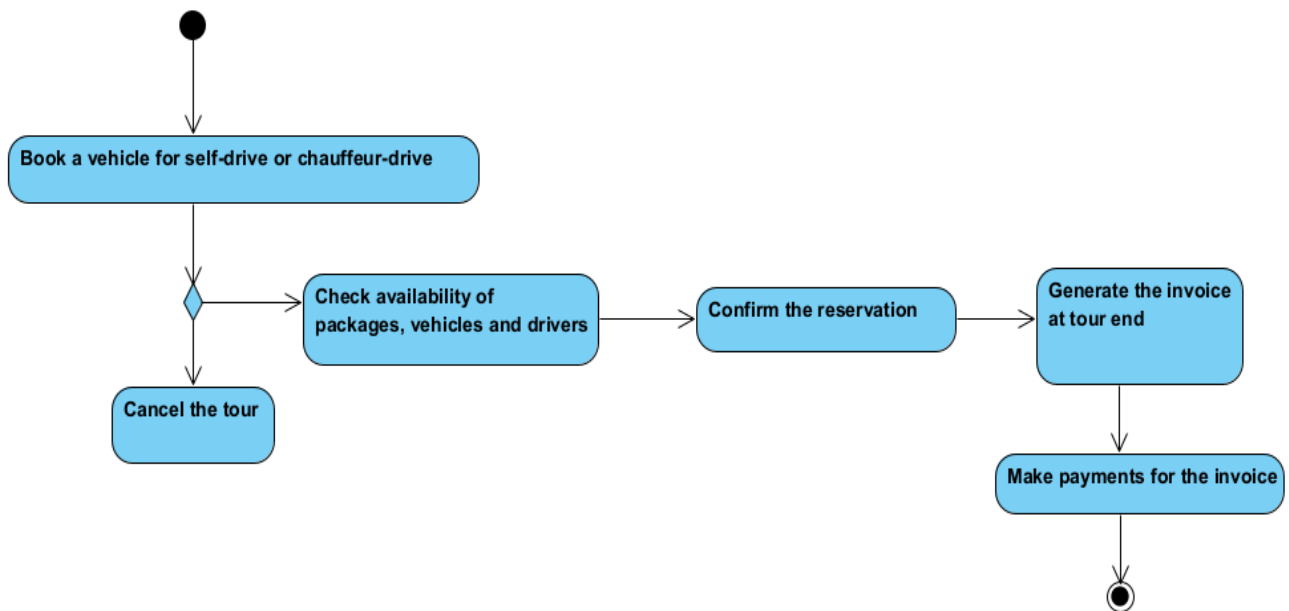


Figure 3. 5: Activity Diagram for Making New Reservation.

Individual ER diagrams for main modules, use case diagrams and narratives, and a few more UML diagrams for the system are given in Appendix B: Design Documentation.

3.5 User Interface Design

The user interface is the primary source of interactions between the user and the system. Good user interfaces are well-structured and user-friendly. Interfaces of the developed system had been designed by following core design principles such as principles of visuals hierarchy and colors. One can get a complete idea about various user interface designs and features implemented in the developed system by referring to the 'user manual' in Appendix C. Several interface designs of the VRMS are mentioned hereafter.

3.5.1 User Login

User login has designed to be simple yet attractive to the user. It welcomes the user into the system. System users can be logged into the system by providing the user name and password. When incorrect details are entered, the system will show the relevant error messages. Users will be taken to the dashboard page when a successful login occurs. The following Figure 3.6 shows the interface of user login of the system.

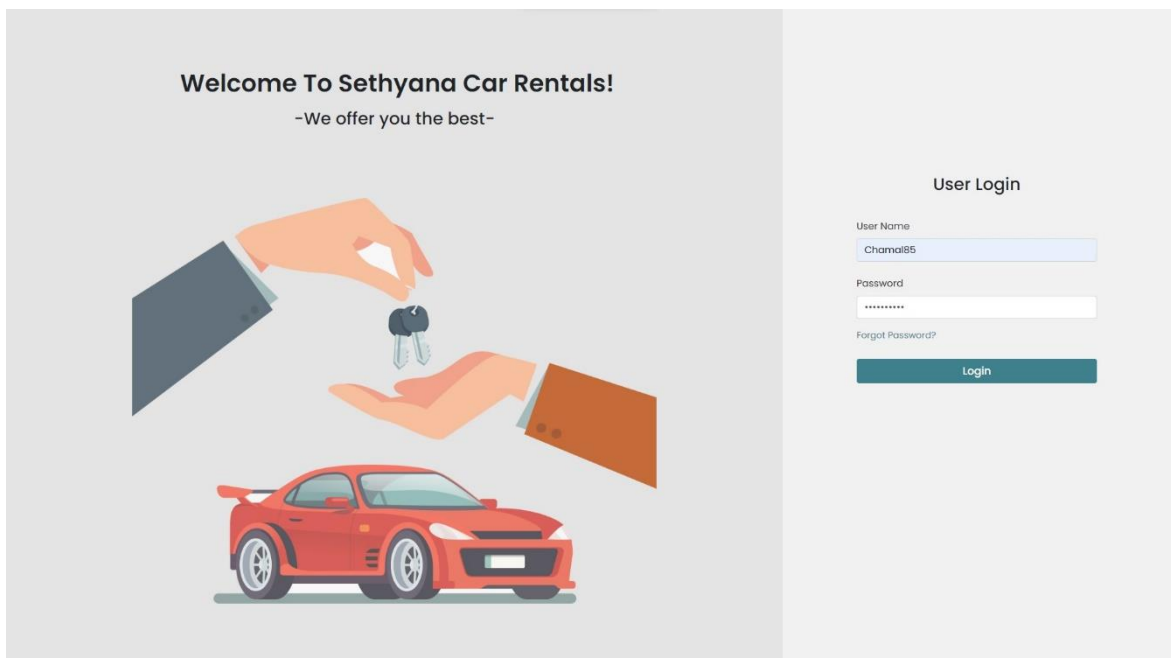


Figure 3. 6: Interface of User Login.

3.5.2 System Dashboard

The system dashboard has been designed with components such as a sidebar, breadcrumb, quick access panel, notification bell, and logged user. System navigation can be performed mainly through the sidebar. The sidebar is designed to be minimized on small screens automatically. Breadcrumbs shows which page the system user is currently at. It also allows users to go one step back by clicking links in breadcrumbs. The quick access panel is designed to be collapsed; therefore, it does not always take the space of the screen. It gives the users access to frequent and essential tasks to perform through the system, like making new reservations, registering a customer, vehicle, etc.

All the system notifications will be shown when the user clicks on the notification bell. Details of the user who is logged into the system can be seen on the nav bar to the top right. When the user clicks on his profile photo, user details and password-changing options will be shown in a dropdown. The following Figure 3.7 shows the interface of the system dashboard.

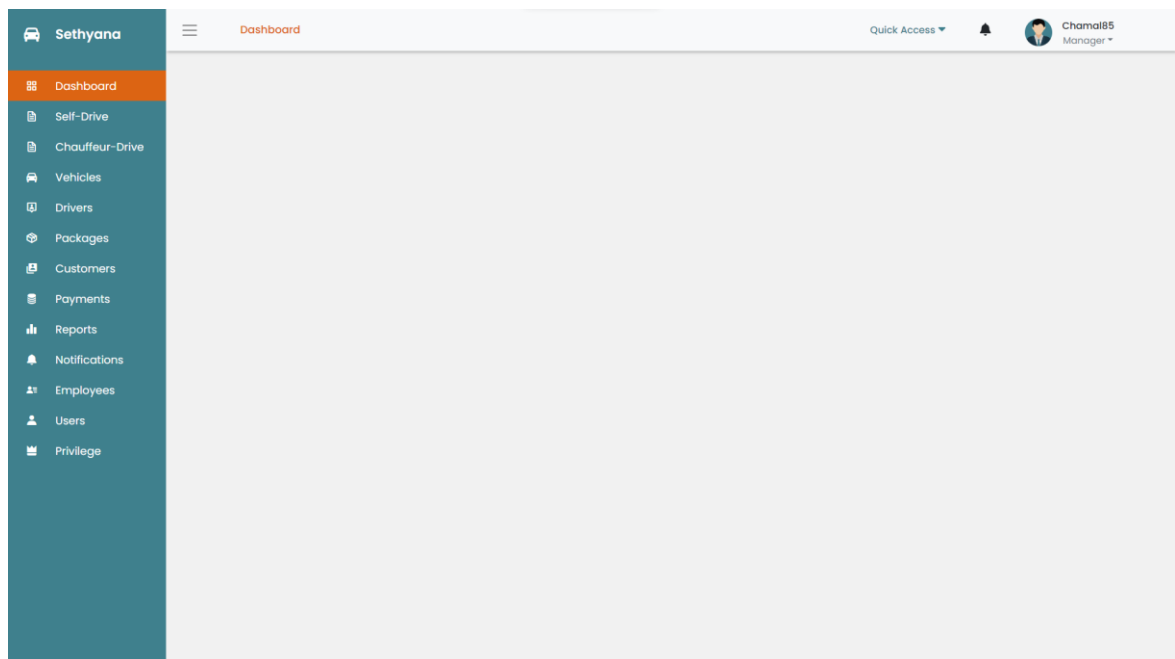
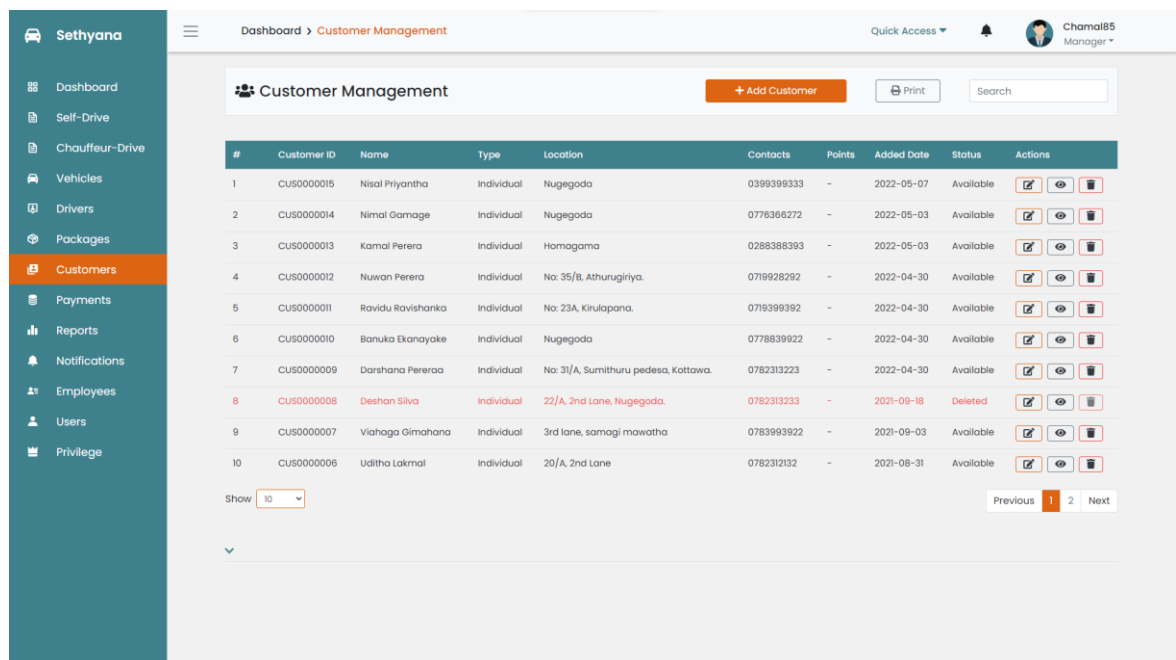


Figure 3. 7: Interface of System Dashboard.

3.5.3 System Table Design

All the system tables have been designed in a similar format so that the user does not get confused from module to module. There is a search bar to type and find records in the table, and a table can be printed using the 'Print' button. All the table records can be updated, viewed, printed, or deleted using the button provided. The table pagination process is carefully designed, and the user can specify how many records to show per page. The following Figure 3.8 shows the table interface of customer management.



Sethyana Dashboard > Customer Management Quick Access Chama185 Manager

Customer Management + Add Customer Print Search

#	Customer ID	Name	Type	Location	Contacts	Points	Added Date	Status	Actions
1	CUS0000015	Nisal Priyantha	Individual	Nugegoda	0399399333	-	2022-05-07	Available	  
2	CUS0000014	Nimal Gamage	Individual	Nugegoda	0776366272	-	2022-05-03	Available	  
3	CUS0000013	Kamal Perera	Individual	Homagama	0288388393	-	2022-05-03	Available	  
4	CUS0000012	Nuwan Perera	Individual	No: 35/B, Athurugiriya.	079928292	-	2022-04-30	Available	  
5	CUS0000011	Ravidu Ravishanka	Individual	No: 23A, Kirulapana.	079399392	-	2022-04-30	Available	  
6	CUS0000010	Banuka Ekanayake	Individual	Nugegoda	0778839922	-	2022-04-30	Available	  
7	CUS0000009	Darshana Pereraa	Individual	No: 31/A, Sumithuru pedesa, Kottawa.	0782313223	-	2022-04-30	Available	  
8	CUS0000008	Deshan Silva	Individual	22/A, 2nd Lane, Nugegoda.	0782313233	-	2021-09-18	Deleted	  
9	CUS0000007	Viahaga Glimahana	Individual	3rd lane, samagi mawatha	0783993922	-	2021-09-03	Available	  
10	CUS0000006	Uditha Lakmal	Individual	20/A, 2nd Lane	0782312132	-	2021-08-31	Available	  

Show 10 Previous 1 2 Next

Figure 3. 8: Table Interface of Customer Management.

The following Figure 3.9 shows the table interface of vehicle management.

#	License Plate	Photo	Model	Vehicle Type	Seats	Odometer	Next Service	Fuel	Owner	Status	Actions
1	ABC-6590		Suzuki Wagon R 2015	Hatchback	5 Seats	55882 Km	2022-08-14	Hybrid	Company	On Rent	
2	CAD-2932		Toyota Prius 2015	Sedan	5 Seats	16100 Km	2022-07-29	Hybrid	Company	Available	
3	CAJ-1453		Honda Fit 2012	Hatchback	5 Seats	19100 Km	2022-08-05	Hybrid	Company	Available	
4	BAC-3884		Mitsubishi Montero 2014	Luxury	7 Seats	18000 Km	2022-12-18	Diesel	Company	Available	
5	CAL-3893		Toyota Vitz 2018	Hatchback	5 Seats	25100 Km	2022-10-06	Petrol	Company	Available	
6	ABC-5899		Suzuki Wagon R 2015	Hatchback	5 Seats	30000 Km	2022-11-09	Hybrid	Company	On Rent	

Figure 3. 9: Table Interface of Vehicle Management.

3.5.4 System Form Design

System forms are critical components that allow users to input data into the system. Lots of thinking went into carefully designing the system forms in order to take accurate user input into the system. At the top of every form, there is a red star indicating required fields. Every field is labeled and given a placeholder indicating helpful information to understand the field to the system user better. Further, tooltips have been embedded into the system forms to explain better what type of data a field might expect. And carefully picked icons have been placed just before the input fields to understand quickly what the input field is about.

Various input fields such as text fields, number fields, date and time fields, combo boxes, radio buttons, toggles and buttons have been implemented in right places in order to get relevant and accurate data from the users. The data validation mechanism has been implemented in input fields and color-coded relevantly. Some forms have grouped similar details with collapse, as shown in Figure 3.10. Also, tab design has been introduced to indicate steps in particular tasks, such as making a new reservation requires certain steps to be followed in order to submit a new reservation, as depicted in Figure 3.11 and Figure 3.12.

Sethyana Dashboard > Vehicle Management

Add Vehicle

Basic Info

License Plate * Vehicle Type * Brand * Model *

Specification

Transmission Type * Air Condition * Fuel Type * Color *

Number of Seats * Luggage Max Weight kg Kilometre per Liter km

Odometer at Vehicle Entry ☐ USB Charging ☐ Rearview Camera

Documents

Revenue License Expire Date * Insurance Expire Date *

Maintenance

Next Service Odometer Next Service Date Damage Update Date

Figure 3. 10: Interface of Vehicle Registration.

Sethyana Dashboard > Chauffeur-Drive Reservations

Add Reservation

Time & Location

Pick Up Date & Time * Pick Up Location * Charge for pick up? * ☐ No ☒ Yes Price Rs.

Drop Off Date & Time Drop Off Location * Have multiple stops? * ☒ No ☐ Yes

Show Previous 1 2 Next

Figure 3. 11: Interface of Chauffeur-Drive reservation form: Time & Location Tab.

Sethyana

Dashboard
Self-Drive
Chauffeur-Drive
Vehicles
Drivers
Packages
Customers
Payments
Reports
Notifications
Employees
Users
Privilege

Dashboard > Chauffeur-Drive Reservations

Quick Access

Chamal85 Manager

Add Reservation

Time & Location

Package

Vehicle

Driver

Customer

Confirm

Summary

Reservation# CDR0000017

Customer : Viahaga Gimahana
NIC : 942452348V
Driver Name : Chaminda Perera
Driving License : B9324942
Vehicle Model : Toyota Vitz 2018
License Plate : CAL-3893

Pick Up Date : 2022-06-20 12:42 PM
Drop Off Date : 2022-06-20 03:42 PM
Pick Up Location : Avissawella
Drop Off Location : Borella
Package Km : 30
Package Duration : 3 Hour

Total Price : 1600.00

Notes

Customer have multiple stops

Reservation Status *

Select Reservation Status

Update Reservation

Add Reservation

Added By : Nipuni Wathsala

Added Date : 2022-06-19 06:11 PM

Updated By : Chamal Wijerathna

Updated Date : 2022-06-28 01:23 AM

Show 10

Previous 1 2 Next

Figure 3. 12: Interface of Chauffeur-Drive reservation form: Confirm Tab.

3.5.5 Reports Generation

System users can generate various rental and revenue reports using the report module of the system. Users can select start and end dates from the date pickers to set the report's time duration. Users also can specify the type of report that needs to be displayed based on day, week, or month. A color-coded bar chart will be generated on the right side of the window, and all the details of the report will be displayed in tabular format to the left. The following Figure 3.13 shows the report generation module.

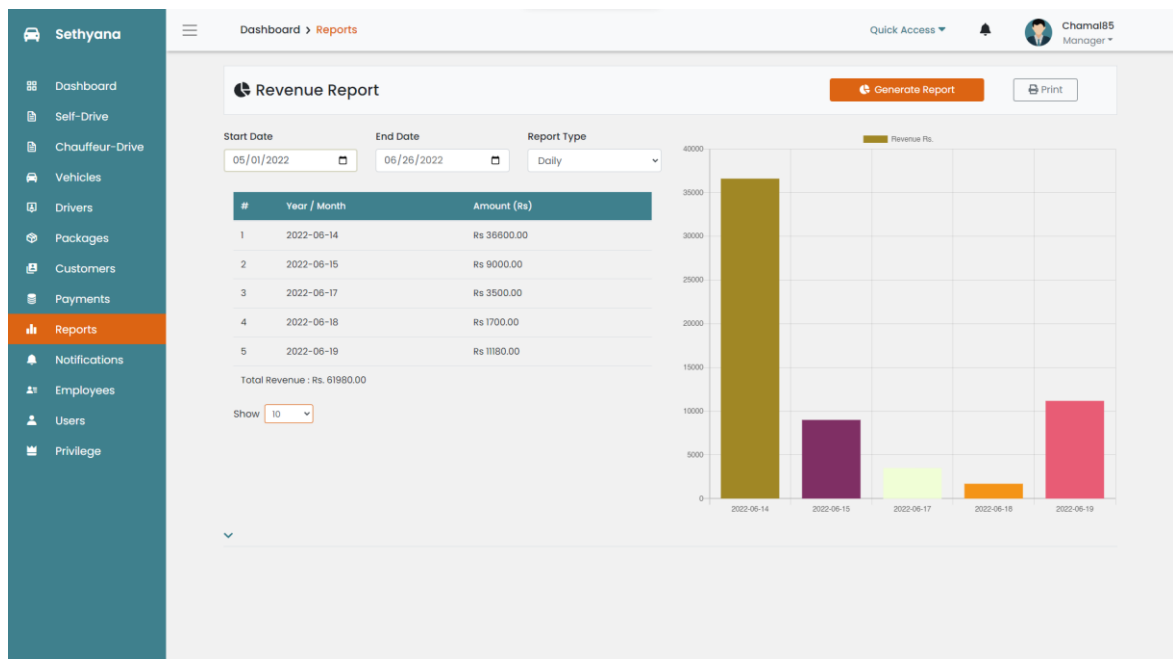


Figure 3. 13: Interface of Report Generation Module.

3.5.6 Invoice Design

The following Figure 3.14 shows a generated vehicle rental agreement for self-drive reservations.



VEHICLE RENTAL AGREEMENT

Reservation No: SDR0000003
Invoice No: PAY0000007
Issued: 2022-06-14 09:59 PM

Customer Details

Name : Rukshan Ruchiraa
Phone : 0778719760
Address : 2/A, 2nd Lane, Samagi Mawatha
Driving License : 98734678
License Expire Date : 2022-06-28 05:30 AM

Vehicle Details

Model : Suzuki Wagon R 2015
License Plate : ABC-6590
Seats : 5
Luggage : 2 Bags
Color : Silver

Package Details

Package Code : PKG0000005
Km Per Day : 100 Km
Price Per Day : Rs 3000.00
Price Per Additional Km : Rs 40.00
Price Per Additional Hour : Rs 360.00
Refundable Deposit : Rs 20000.00

Reservation Details

Pick Up Location : Kaduwela
Return Location : Office
Pick Date & Time : 2022-06-04 11:14 PM
Return Date & Time : 2022-06-09 01:58 PM
Rental Days : 4 Day
Additional Hours : 14 Hour
Odometer Out : 55882 Km

Charge Details

Rental Days Charge : Rs 12000.00
(4 Day X Rs 3000.00)
Additional Hour Charge : Rs 5040.00
(14 Hour X Rs 360.00)
Pick Up Charge : Rs 0.00
Total: Rs 17040.00
Paid Advance : Rs 9000.00
Due Balance : Rs 8040.00
Refundable Deposit : Rs 20000.00 Paid

I HAVE READ AND AGREED TO THE TERMS AND CONDITIONS ON **BOTH SIDES** OF THIS AGREEMENT.

.....

Customer Signature

.....

Authorized By: Admin

Place Official Seal

THANK YOU FOR CHOOSING SETHYANA CAR RENTAL !

Figure 3. 14: Generated Rental Agreement.

Chapter 4 – Implementation

The system is developed following the guidelines of the incremental and iterative process model. Based on each iteration's design plan, the system is developed and delivered as an improved software product. MVC architecture is followed since it helps to write more organized, testable, and maintainable code. The system implemented environment and tools and technologies used in this project are discussed, along with the reasons and advantages of picking particular ones. System code and module structure are explained, providing the actual code of the project. Finally, reused codes taken outside are mentioned at the end of the chapter.

Various types of tools and technologies have been used in the implementation phase. Java language has been used to develop the back end of the system. Spring boot and hibernate frameworks have been used since they avoid repetitive codes and support rapid development. Furthermore, HTML, CSS, Bootstrap, and JavaScript were used to develop the system user interfaces and logic.

4.1 Implemented Environment

When choosing software tools and technologies and other resources, factors like business requirements, development cost and time, scalability and performance, maintainability and setup costs, and user-friendly tools are considered. The following Table 4.1 shows hardware and software requirements for the developed software system.

Hardware	Software
• Processor – Intel Core i5 2.40 GHz	• Microsoft Windows 10
• RAM – 8GB	• Java 11
• Hard Disk – 500 GB	• MySQL 8
• Printer	• MySQL Workbench v8.0
	• Apache Tomcat 8.5
	• Google Chrome v95.0

Table 4.1: Hardware & Software Requirements

4.2 Tools and Technologies Used

4.2.1 Java Language

Java is one of the most widely used high-level programming languages out there. It is a simple and stable programming language with massive community support. One of the main advantages of Java is that it supports object-oriented programming, which provides many benefits compared to procedural languages, which are very complicated and hard to implement. In addition, Java uses OOP concepts such as encapsulations, inheritance, and abstraction, which increase security, unauthorized access, and the reusability of the code. Furthermore, Java is platform-independent, making it a highly portable language. Java automatically manages the memory efficiently using garbage collection, which removes objects that do not refer to any classes, so developers do not need to write any extra code [10]. Ultimately, it was chosen to develop the back end of the VRMS with Java 11, which is a more stable version of Java to this day as writing this document.

4.2.2 MySQL

MySQL is a Relational Database Management System (RDBMS) that stores records in multiple tables instead of in a single repository [11]. A couple of considerations went into selecting MySQL to handle the database of VRMS. First, MySQL is globally known for being one of the most reliable and secure database management systems. It protects sensitive data from intruders, and passwords in MySQL are saved after encryption. Second, MySQL ensures optimum speed, full-text indexes, and unique memory caches for enhanced performance. Views, triggers, and stored procedures in MySQL help to increase the productivity. Another reason was to choose MYSQL that its availability. MySQL ensures 24x7 uptime offering a wide range of high-availability solutions. In addition, MySQL is open-source and backed by a large community of programmers, which means that codes are well tested, and most bugs are fixed quickly. Considering all those factors, let alone being simple and easy to learn and use, it has been the go-to solution to handle the database of the VRMS.

4.2.3 Spring Framework

Spring framework has been used in developing the VRMS. In the first place, why use any framework? A framework helps developers focus more on the core task rather than the boilerplate associated with it, which speeds up the development process. A good framework is integrated with good design patterns and adheres to industry standards. Further, it cuts down the total cost of ownership of the software system. There are a few reasons to choose the Spring framework in developing the proposed system. First its usability. Spring promotes convention over configuration, making it easier for programmers to start and configure exactly what is necessary. In other words, its modular nature gives options to use the entire Spring framework or just start with what is needed and then include more spring projects on the go. When selecting any framework, support for testing should be a concern. Spring at the core supports Test Driven Development (TDD) [12]. Spring has massive community support, and its maturity makes it a default solution for most common problems.

4.2.4 Hibernate

Hibernate is an open-source, lightweight object-relational mapping (ORM) tool for java applications to interact with the database. Cash being internally used in Hibernate provides good performance. Hibernate Query Language (HQL), which is the object-oriented version of SQL, generates database-independent queries. Creating tables and fetching data from multiple tables is fairly easy in Hibernate framework [13]. Furthermore, Hibernate addresses the object-relational impedance mismatch issue by replacing direct persistence-related database access with high-level object handling functions. [14].

4.2.5 Apache Tomcat

Apache Tomcat is an open-source implementation of many Java enterprise specs such as Java Server Pages, Java Servlet, Java Expression Language, and WebSocket technologies. Besides being a lightweight application, it provides all the necessary and standard functionalities required to operate a server. It can give fast load and redeploys compared to its various alternatives [15]. Tomcat is one of the most stable and popular application servers used for java web-based applications.

4.2.6 Gradle

Gradle is an open-source build automation system based on the concepts of Apache Ant and Apache. It provides support for building, testing, and deploying software on different platforms, including Java. It also provides integration with IntelliJ IDEA, which is the chosen IDE for VRMS development.

4.2.7 IntelliJ IDEA

IntelliJ IDEA is a powerful and popular integrated development environment (IDE) for Java. It is developed and maintained by Jet Brains Company and licensed by Apache 2.0. It supports build tools like Gradle and Maven and comes with a built-in decompiler for Java classes. It supports version controllers like Git and web servers like Tomcat. IntelliJ provides database tools to connect to live databases and manipulate data. With IntelliJ, unit testing can be done with ease. IntelliJ has a less learning curve, and the user experience is much more satisfying [16]. With no arguments, it is a perfect fit as an IDE for previously determined technologies in developing the VRMS.

4.2.8 MySQL Workbench

MySQL Workbench is a visual and graphical tool used to design, model, generate and manage databases. It helps developers to create intricate ER models, perform forward and reverse engineering and also aids in change management and documentation tasks that often take a lot of time and effort. MySQL Workbench provides visual tools to create, execute, and optimize SQL queries. SQL editor delivers features such as query auto-complete, syntax highlighting, reuse of SQL snippets, and execution history of SQL. Furthermore, the Database Connections Panel allows developers to simply manage database connections. [17].

4.2.9 Visual Paradigm

Visual Paradigm is a powerful, feature-rich, and easy-to-use UML CASE tool. It delivers many features; model-driven development support, extensive database coverage, IDE integration, object-relational mapping, and database reverse and forward engineering, to mention a few [18]. However, this tool is used mainly to design and draw UML diagrams for the VRMS.

4.3 Code and Module Structure

4.3.1 MVC Architecture

The system is developed on top of the MVC framework. MVC framework is a software developing architecture that separates an application into three main logical components; the model, the view, and the controller. This promotes modularity and ease of collaboration and reuse, and it also makes the application more flexible. MVC is one of the most frequently used web development frameworks in the industry to create scalable and extensible projects.

Model – The model component stores all the data-related logic that the user works with. It represents the data that is being transferred between the view and controller components or any other business logic-related data. The model responds to the requests from the view as well as the instructions from the controller to update itself.

View – The view component represents all the user interface logic of the application. The view must make sure that its appearance reflects the state of the model. Whenever the data changes in the model, it notifies the views that depend on it through the controller.

Controller – The controller component functions as an interface between the model and view components. It handles user requests coming from the view component and then manipulates and processes data using the model component and updates the view as appropriate. Following Figure 4.1 shows the main components of the MVC architecture.

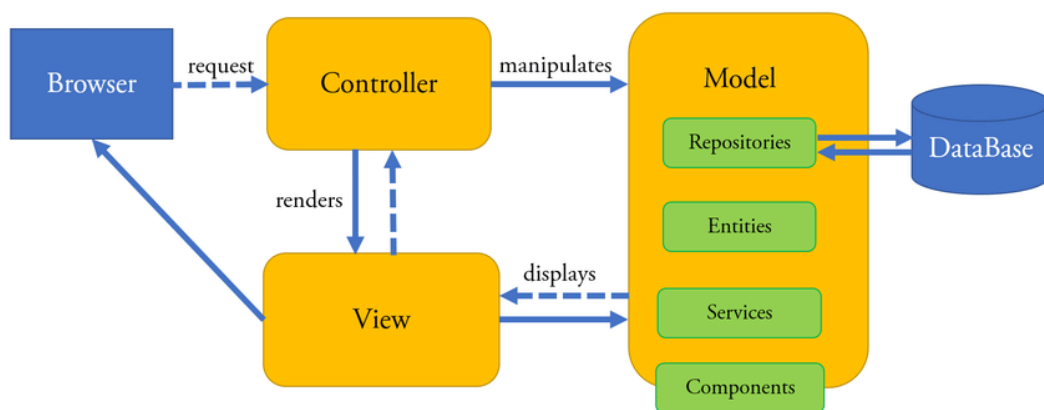


Figure 4.1: Diagram of MVC architecture [19].

4.3.2 The Model Implementation

The model contains the application's data and exists as a Plain Old Java Object known as POJO. In fact, data models are made up of entities, and they become the tables in a database, and entities have attributes that are mapped to columns in a database table.

`@Entity` annotation specifies that the class is an entity mapped to a database table, and `@Table` annotation specifies the name of the database table to be mapped. `@Data` annotation generates all the boilerplate that is usually associated with Java objects, including setters and getters. `@NoArgsConstructor` generates an empty constructor, while `@AllArgsConstructor` generates a constructor with one parameter for every field in the class. `@Column` maps the column in the database table with the field.

`@JoinColumn` annotation specifies a column for joining an entity association. After then the `@ManyToOne` annotation is used to define a many-to-one relationship between those two entities which are connected with the `@JoinColumn` annotation.

Data validation is a critical aspect of system security. Suppose a user could enter any values with a particular input. In that case, the system can be corrupted, or the user may gain unwanted access to the system via unethical behaviors like hacking. Therefore, to prevent such scenarios, 'regular expressions' has been used to validate user input in VRMS strictly. `@Pattern` annotation can be used to achieve regular expression validation in Spring MVC. The following code segment shows the model implementation for the customer module.

```

@Entity //identify as a model
@Table(name = "customer") //map with database table
@Data //setters & getters
@NoArgsConstructor //empty constructor
@AllArgsConstructor //all arguments constructor
public class Customer {

    @Id //primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) //auto increment
    @Basic(optional = false) //not null
    @Column(name = "id") //map with database column name
    private Integer id;

    @Column(name = "register_no", unique = true)
    @Basic(optional = false)
    @Pattern(regexp = "[0-9]{10}$", message = "Invalid Register Number")
    private String register_no;

    @Column(name = "photo")
    @Basic(optional = true)
    private Byte photo;

    @Column(name = "first_name")
    @Basic(optional = true)
    @Pattern(regexp = "[a-zA-Z\\'.-]+$", message = "Invalid First Name")
    private String first_name;

    @Column(name = "nic")
    @Basic(optional = true)
    @Pattern(regexp = "[0-9]{9}[vV][0-9]{12}$", message = "Invalid NIC Number")
    private String nic;

    @JoinColumn(name = "customer_type_id", referencedColumnName = "id")
    @ManyToOne(optional = false, fetch = FetchType.EAGER) //many to one relationship
    private CustomerType customer_type_id;

```

4.3.3 The Controller Implementation

The controller is a part of the application responsible for handling user requests and informs the model and view to change as appropriate. In fact, the controller sends commands to the model and updates its state, and also sends commands to its associated view in order to render the user interface.

`@RestController` annotation is a convenient annotation that combines `@Controller` and `@ResponseBody` annotations, and it simplifies the creation of RESTful web services. This annotation marks the class as a request handler that takes care of mapping request data. `@RequestMapping` annotation maps HTTP requests to handler methods of MVC and REST controllers. `@Autowired` annotation has been used on properties to instantly inject objects into a relevant property on run time. `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping` have been used to map Get, Post, Put, and Delete HTTP requests to relevant handler methods, respectively.

When an HTTP request is sent, mapping annotation maps the relevant handler method according to the HTTP request. Then it calls the `getAuthentication()` function to retrieve the security authentication object. Then check if the currently logged user exists in the database and has privileges to the module. If all those checks are passed, it interacts with the repository and performs the required actions with the database. Moreover, 'try' and 'catch' methods have been placed to handle the exceptions when performing CRUD operations on the database.

The following code segment shows the controller implementation for the customer module in VRMS. Important code segments are commented for ease of understanding.

```

@RestController //make controller class readable to servlet
@RequestMapping(value = "/customer") //url mapping
public class CustomerController {

    @Autowired
    private UserService userService;

    @Autowired
    private PrivilegeController privilegeController;

    @Autowired // create an instance of an interface
    private CustomerRepository dao; //declare interface type variable/ dao-data access object
    @Autowired //create customerStatus interface instance(object)
    private CustomerStatusRepository daoStatus;

    //Get Mapping: for individual list
    @GetMapping(value = "/list_individual", produces = "application/json")
    public List<Customer> customersListIndividual() {
        return dao.listIndividual();
    }

    //Get Mapping: for Customer List with Searched Text
    @GetMapping(value = "/findAll", params = {"page", "size", "searchtext"}, produces =
"application/json")
    public Page<Customer> findAll(@RequestParam("page") int page, @RequestParam("size")
int size, @RequestParam("searchtext") String searchtext) {
        //get security context authentication for logged user
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        // get logged username and find in db
        User user = userService.findUserByUserName(auth.getName());
        //Retrieve which privileges the user has for module
        HashMap<String, Boolean> priv = privilegeController.getPrivileges(user,
"CUSTOMER");

        //check user null //check priv null // get privilege for search (findAll)
        if (user != null && priv != null && priv.get("select")) {
            return dao.findAll(searchtext, PageRequest.of(page, size, Sort.Direction.DESC, "id"));
        } else {
            return null;
        }
    }
}

```

Repository Implementation

The repository pattern has been used with the MVC architecture in the development of this project. It creates an abstraction layer between the data access layer and the business logic layer of the software system, which encapsulates data storage, retrieval, and search. Repository is an interface that extends the 'JpaRepository'. JpaRepository is a specific extension of the repository of Java Persistence API (JPA). It includes API for basic CRUD operations and also for pagination and sorting, which makes the process of working with entities a lot easier. @Query annotation has been used to define SQL to execute for a Spring Data repository method. Its value attribute accepts both JPQL or SQL to be executed.

The following code snippet shows the repository implementation of the customer module.

```
//extend JpaRepository from Spring for more functionalities for crud
public interface CustomerRepository extends JpaRepository <Customer, Integer> {

    @Query("select c from Customer c where (c.register_no like concat('%', :searchtext, '%') or
" +
    "c.first_name like concat('%', :searchtext, '%') or c.last_name like concat('%',
:searchtext, '%') or " +
    "c.company_name like concat('%', :searchtext, '%') or c.customer_type_id.name like
concat('%',:searchtext, '%') or " +
    "c.phone like concat('%', :searchtext, '%') or c.company_phone like concat('%',
:searchtext, '%') or c.contact_person_phone like concat('%', :searchtext, '%') or " +
    "c.email like concat('%', :searchtext, '%') or c.company_email like concat('%',
:searchtext, '%') or c.customer_status_id.name like concat('%', :searchtext, '%'))")

    //Query Returning Function Mapping
    //@Param to insert searchtext value to @Query
    Page<Customer> findAll(@Param("searchtext") String searchtext, Pageable of);

    @Query("select c from Customer c where c.customer_type_id='1'")
    List<Customer> listIndividual();
}
```

4.3.4 The View Implementation

View layer is responsible for data representation. View generates the relevant user interface with the data being passed through the model to the controller. View only communicates with the controller. To alter the user interfaces, technologies like HTML, CSS, Sass, Bootstrap, JavaScript, and jQuery have been used in the view implementation. Code segments mentioned under the sub-headings of Imports, Navbar, Table and Form show the view implementations for the customer module.

Table

System table is designed to show important details of a record in tabular format. Buttons are integrated into the table to perform CRUD operations on each record. The following code segment shows the table implementation of the view layer for the customer module.

```
<!--TABLE-->
<section>
  <div class="container">
    <div class="row p-3 formcontent" id="divTable">
      <div class="table-responsive" class="table-container ml-2" id="tableParent">

        <script>
          function getCustomerName(ob) {
            var name = "";
            if (ob.customer_type_id.name == "Individual") {
              name = ob.first_name + " " + ob.last_name;
            } else {
              name = ob.company_name;
            }
            return name;
          }

          var metadata = [
            <!--name:table columns / property:column name-->
            {name: 'Customer ID', search: false, datatype: 'text', property: 'register_no'},
            {name: 'Name', search: false, datatype: 'text', property: getCustomerName},
            {name: 'Type', search: false, datatype: 'text', property: 'customer_type_id.name'},
```


Form

System forms are designed to open in a modal. Form implementation of the view layer for the customer module is shown in following code segment.

```
<!-- FORM MODAL -->
<div class="modal fade" id="addCustomerModal" role="dialog" tabindex="-1">
  <div class="modal-dialog modal-xl" role="document">
    <div class="modal-content bg-background-color">
      <div class="modal-header bg-primary">
        <h5 class="modal-title ml-auto text-white" id="addCustomerModalLabel"
          style="font-weight: 400; font-size: 1.6rem;"><i class="fas fa-user-plus mr
2"></i>Add Customer
        </h5>
        <button class="close" data-dismiss="modal" type="button">
          <span aria-hidden="true" class="text-white"><i class="fas fa-times"></i></span>
        </button>
      </div>
      <div class="modal-body">
        <!--FORM-->
        <div class="row p-3" id="divFrom">
          <div class="col-md-12">
            <form class="" id="customerForm" novalidate onsubmit="return false">
              <!--onsubmit-false: prevent form refreshing on submit otherwise impossible
to check null pass -->

              <!--Required Mark-->
              <div class="row">
                <div class="col-md-12 text-right">
                  <p class="m-0 text-danger" style="font-weight: 400;"><span
                    class="required">*</span>
                    Required</p>
                </div>
              </div>
            </div>
          <div class="col-md-3">
```

4.4 Reused Existing Codes and APIs

While developing the system, the following reusable components have been used. Some of the components are implemented to increase user friendliness and achieve maximum effectiveness through well-defined reusable functions.

Pagination

The following customized JavaScript code segment has been used to create system pagination.

```
function createPagination(paginationid,pagecount,activepage, paginatefunction){
    var pagination = document.getElementById(paginationid);
    pagination.innerHTML="";

    var lipre = document.createElement('li');
    lipre.classList.add('page-item');
    lipre.classList.add('previous');
    var apre = document.createElement('a');
    apre.classList.add('page-link');
    apre.innerHTML = "Previous";
    apre.onclick = function(){
        paginatefunction(parseInt(activepage)-1);
    }

    lipre.appendChild(apre);
    pagination.appendChild(lipre);

    for(i=1;i<=pagecount;i++) {
        var li = document.createElement('li');
        li.classList.add('page-item');
        var a = document.createElement('a');
        a.classList.add('page-link');
        a.innerHTML = i;
        if(i!=activepage) {
            a.href = "#";
            a.addEventListener("click", function () {
```

Side Bar

The following CSS code segment has been used to create the sidebar, especially to achieve the collapsing behavior. Note that it has been customized to match the look and feel of the system interface.

```
/* SIDEBAR */
.sidebar{
  position: fixed;
  top: 0;
  left: 0;
  height: 100%;
  width: 260px;
  background: var(--primary-color);
  z-index: 100;
  transition: all 0.5s ease;
}

.sidebar.closes{
  width: 78px;
}

.sidebar .logo-details{
  margin-top: 11px;
  height: 60px;
  width: 100%;
  display: flex;
  align-items: center;
}

.sidebar .logo-details i{
  font-size: 30px;
  color: #fff;
}
```

Other Used Components

- SweetAlert library has been used to implement some of the system's validation messages.
- To achieve type and search functionality for the select combo box, Select2 component has been used.

Chapter 5 – Evaluation

Testing is the process of evaluating a system and its component with the intention to find if the developed system satisfies the specified requirements or not. It is an essential process of software development in order to find bugs and errors in the developed system and solve them. Software testing, in other words, is a method of verification and validation.

5.1 Testing Strategies

5.1.1 Unit Testing

Unit testing is a software testing methodology that involves testing each unit or an individual component of the software. It is the first level of functional testing [20], and it not only validates the system components but also improves performance. In addition, performing unit testing increases the code's robustness, reducing the regression in future development and maintenance.

During this project, unit testing has been performed at end of the development of each module using JUnit and Gradle build tool. Test classes are separated from the main source code located in 'src/main/test' directory. Further, standard package and test case naming conventions have been used with JUnit.

5.1.2 Integrated Testing

In integration testing, individual components are combined and tested as a group. It is the second level of software testing, which comes after unit testing and before validation testing [21]. Integration testing helps to expose defects when system components are integrated. During this project, modules used for unit testing are combined and tested for integration testing.

5.1.3 System Testing

In system testing fully integrated software system is tested as a whole in similar real business environment to evaluate the system's compliance with client's specified

requirements. Components that are passed integration testing are taken as input in system testing [22]. System testing is a black-box testing method that aims to detect any irregularity between the integrated units while all the peripherals are connected to the system.

During this project, the following types of system testing were carried out.

- Performance testing is carried out to test the system's speed, stability, and reliability.
- Load testing is carried out to determine the behavior of the system under heavy load.
- Usability testing is carried out to determine the user's ease of using the application and flexibility in handling the system.
- Regression testing is carried out to make sure that no new bugs have been introduced over the course of development and that old bugs do not appear after new modules have been implemented into the system.
- Migration testing is done to ensure that the system is working without introducing new issues after moving from old infrastructure to current infrastructure.

5.2 Test Plan

Creating a test plan is a crucial aspect of developing a system to ensure that the system will be tested properly and minimize any defects. The test plan consists of detailed information about testing procedures, including testing scope, testing schedule, testing deliverables, etc. A proper test plan is capable of testing the efficiency of the overall system.

The implemented system was tested using various test cases. At the end of the development of each module, unit testing was carried out so that errors in each module could be identified earlier. After developing all the modules required to complete a business scenario, then those modules were combined and tested for integration test.

At the completion of the development of the system, system testing is carried out to assess the system performance as a whole in a similar real business environment.

5.3 System Test Cases

A test case is a test scenario measuring and verifying a particular feature or functionality of the software application. Properly planned test cases are capable of verifying and validating the functionality of a system component. In order to reduce the complexity, the system was divided into modules, and test cases were written for each module. Additionally, test results of some of the important test cases, along with some screenshots, have been added to Appendix E. The following test cases contain the test number, test description, steps to test, and the expected results.

5.3.1 Test Cases for Customer Module

The following Table 5.1 shows the test cases for the customer module.

Test No.	Test Description	Steps to Test	Expected Result
1	Generate a new customer ID	Click 'Add Customer' button	Interface of the customer form should load up with autogenerated Customer ID.
2	Default values on load	Click 'Add Customer' button	Customer Status: 'Available' Added By: Current logged-in employee Added Date: Current Date
3	Data validation (negative)	Enter invalid data into the field.	Display an appropriate error message and switch to the border of the input field red.
4	Data validation (positive)	Enter valid data into the field.	Border of the input field is switched to green and displays a green tick next to the field.
5	Add a customer (negative)	Leave the required field blank and click 'Add' button.	Display an error message indicating missing the required fields.
6	Add a customer (positive)	Fill in all the required fields and click 'Add' button.	Display the message "save successfully."
7	Manage existing customer details	Click update/ view/ search according to	Customer details should be able to be updated and

		relevance.	view/search according to the customer ID.
8	NIC validation	Type values into the field.	Validate only if the series of entered values contains nine digits and ends with v/V or the series contain 12 digits.
9	Driving license validation	Type values into the field.	Validate only if series of entered values contains seven digits, begins after uppercase B.
10	Phone number validation	Type values into the field.	Validate only if a series of entered values contains ten digits in total and begins with zero.

Table 5.1: Test Cases for Customer Module.

5.3.2 Logging Related Test Cases

Following Table 5.2 shows the login-related test cases.

Test No.	Test Description	Steps to Test	Expected Result
1	Log in (negative)	Enter invalid username or/ and password.	Display error message "Username or Password invalid, please try again"
2	Log in (positive)	Enter valid username and password.	Validate and redirect to the dashboard.
3	Login time out	Idle for 20 minutes.	Logout user
4	Log out	Click on user name on top navbar and select log out	Display confirmation message "Do you want to log out?" If click "Yes", redirect to Log in page.

Table 5.2: Login Related Test Cases.

5.3.3 Test Cases for Package Module

Following Table 5.3 shows the test cases for package module.

Test No.	Test Description	Steps to Test	Expected Result
1	Load the package-form	Click 'Add Package' button	Package-form interface should load up with autogenerated package number and the inner table of vehicle model.
2	Default values on load	Click 'Add Package' button.	Customer Status: 'Available' Added By: Current logged in employee Added Date: Current Date
3	Data validation (negative)	Enter invalid data into the field.	Display the appropriate error message with border of the input field in red.
4	Data validation (positive)	Enter valid data into the field.	Border of the input field becomes green and displays a green tick next to the field.
5	Add a package (negative)	Leave the required field blank and click 'Add' button.	Display an error message indicating missing required fields.
6	Add a package (positive)	Fill in all the required fields and click 'Add' button.	Display the message "save successfully."
7	Manage existing package details	Click update/ view/ search according to relevance.	Package details should be able to update and view/search according to the package number.
8	Package type altering (Self-drive)	Select 'self-drive' package type from the combo box.	Display the 'price per day' field & 'refundable deposit' field by replacing the 'package price' field & 'price per additional hour' field

9	Package type altering (Chauffeur-drive)	Select the 'chauffeur-drive' package type from the combo box.	Display the 'package price' field & 'price per additional hour' field by replacing the 'price per day' field & refundable deposit' field.
10	Search vehicle models	Click on the 'vehicle model' combo box and type the model's name.	If the model exists, system should filter all the models and display on the combo box user to select.
11	Add vehicle models to the package.	Click and search a vehicle model, and click add button.	Selected model must be added to the inner table.
12	Delete vehicle models from the package	Click on the delete button (trash-bin icon) on the preferred record.	Display confirmation message. If hit ok, the model must be deleted from the inner table.

Table 5.3: Test Cases for Package Module.

5.3.4 Test Cases for Common Functions

Following Table 5.4 shows the test cases for common functions.

Test No.	Test Description	Steps to Test	Expected Result
1	Access modules	Click on a module name displayed on the sidebar.	System must load the relevant module interface.
2	Quick Access	Click on quick access link on top navbar.	Collapsible window pane should be displayed with quick access items.
3	Test update, view, delete buttons on a delete record.	Try to click on update, view, delete buttons on a deleted record.	View button and delete button should be disabled on a deleted record and edit button should only be active for the privileged user.
4	Search for a record by ID	Type the ID on the search bar.	If the record exists for given ID, the record must be displayed in the table.

5	Update a record	Click the update button (pencil icon).	Load the interface & form fields with data. If updates are successfully saved, the updated record should be highlighted on the table.
6	Update a record (While the previous update is not saved.)	Click the update button with a pencil icon.	Display confirmation message “Are you sure to discard the form?”
7	Delete a record	Click the delete button (trash-bin icon) on table.	Allow the user to delete after confirming the request. If the delete is successful, the deleted row should be highlighted in red.
8	View details of a record	Click on view button (eye icon) on table.	Detailed information for the record should be appeared in a model.
9	Print a record	Click on view button & click on print button in the opened model.	Open the print dialog of the browser in a new tab with record details.
10	Print all records	Click on “Print” button	Browser print dialog should be displayed in a new tab with records.
11	Load more records	Select number of rows from the ‘Show’ dropdown.	Table should be loaded with more available records.
12	Load Pagination	Load a table.	If table has more records than currently shown on the page, the next relevant page number should be displayed. Previous & Next button should be activated accordingly.
13	Clear all the fields	Click clear button.	The form fields must be cleared.
14	Allow date selection	Click date picker.	Popup calendar to select correct date.
15	Display help popup	Hover mouse over ‘?’ icon	Display information on a

		next to a form-field label.	tool tip.
16	Change user password	Click on user name on top navbar and select change password.	Allow to change password after the correct old password is typed.
17	Test database connection	Type incorrect DB user name and password in application.properties file.	Display correct DB error message.
18	Test system in major browsers	Load the system in different browsers (Chrome, Firefox, Edge, IE)	The system should behave and display its interfaces in similarly in all browsers.
19	Sidebar on small screens	Load the system on small screens.	Should be able to collapse and show the mini sidebar.
20	Tables on small screens	Load the tables on small screens.	Horizontal bar should be activated to scroll horizontally.

Table 5.4: Test Cases for Common Functions.

5.4 User Evaluation

After implementing the system in the office environment, acceptance testing was carried out with real data sets to ensure that the specified business needs of the company were met. After a quick introduction to the system and its functionalities, selected users could access the system with relevant privileges to perform their day today transactions using the newly implemented system. Most of the functions were tested out a few times since different users would perform the same operation. The final result of the test indicated that the system is easy to understand and learn and more user-friendly. The manager was pleased with the system's performance, how easy and guided the reservation-making process was, and the overall user interfaces design. Minor modifications were suggested; however, all the staff members and manager were pleased and satisfied with the delivered system as they could smoothly run their business operations.

Following Figure 5.1 and Figure 5.2 shows the completed user evaluation forms by the manager and the receptionist of the Sethyana company. The certificate received from the client has been attached in appendix F.

User Evaluation Form

Name	A.P. Chamal Wijerathna
Designation	Manager

Vehicle Rental & Reservation Management System for Sethyana Rent a Car & Cab Service					
No	Criteria	Ratings			
		Bad	Average	Good	Very Good
01	Vehicle Management				✓
02	Reservation Mangement			✓	
03	Package Management			✓	
04	Customer Management			✓	
05	System navigation			✓	
06	User interface & color schemes				✓
07	User friendliness				✓
08	Ease of learning			✓	
09	Response time			✓	
10	System security			✓	
11	System accuracy			✓	
12	Supportiveness for management			✓	
13	Level of achievement of stated objectives			✓	
14	Level of satisfaction towards the system			✓	

Suggestions for improvements:

.....25/04/2022.....
Date

Figure 5.1: User Evaluation Form-1

User Evaluation Form

Name	K.M.G. Chinthana Kulathunga
Designation	Receptionist

Vehicle Rental & Reservation Management System for Sethyana Rent a Car & Cab Service					
No	Criteria	Ratings			
		Bad	Average	Good	Very Good
01	Vehicle Management				✓
02	Reservation Management				✓
03	Package Management			✓	
04	Customer Management			✓	
05	System navigation			✓	
06	User interface & color schemes				✓
07	User friendliness				✓
08	Ease of learning				✓
09	Response time			✓	
10	System security			✓	
11	System accuracy			✓	
12	Supportiveness for management			✓	
13	Level of achievement of stated objectives			✓	
14	Level of satisfaction towards the system			✓	

Suggestions for improvements:

...25/04/2022.....

Date

Figure 5.2: User Evaluation Form-2

Chapter 6 – Conclusion

6.1 The Final Evaluation

‘Sethyana Rent a Car and Cab Service’ is a well-known car rental company in Avissawella area. They previously had a requirement to enhance their business with the help of modern technology due to various issues they faced with manual paper-based work ethics. After implementing the developed system, their productivity has increased. Further, the company is being able to clearly identify the cash flow of the business as system-generated reports clearly indicate the revenue and expenses of the business. Making a new reservation is now just a matter of a few clicks from finding out if a suitable package, vehicle, and driver are available. This benefits both the customer and the staff members by saving their valuable time. Now that the staff members are motivated than before, to work through the implemented system since the day-to-day operations are well managed and run smoothly.

Functional and non-functional requirements were identified and implemented for each module. Additional functions and customized features were introduced at the end of the project, which was requested by the client. Further, the customer was eventually pleased with the developed system, which is one significant milestone to measure the success of the project which it was undertaken for.

The completed system was developed in an incremental fashion that has provided the flexibility for the project to be altered according to the changing requirements, which would come ahead of time. One of the industry’s robust programming languages, Java, was used as the primary programming language, whereas MySQL was used as the database server.

6.2 Future Enhancement

Some suggestions for improvements to the system in the future can be listed as follows;

1. Integrate a vehicle tracking system using GPS technology.
2. Purchase Google map API service to easily select pickup and drop-off locations stated by the customer.
3. Introducing online reservations to the customer.
4. Introducing an online payment facility.
5. Integrate more reports into the system.
6. Improve the responsiveness of the current system on mobile devices or develop an android based mobile application.

6.3 Lessons Learnt

As a BIT student following one of the competent degree programs out there, the final project was the most important piece of work in the final year. It was a good opportunity to apply knowledge learned through the first and second years of the degree to ultimately develop a real-world working software application. The domain area was considerably large; however, it broadened my horizons in understanding how those related business processes are mapped into a computerized system.

Communication skills have been polished since it was required to conduct interviews and discussions with the owner to staff members of the company. And also, observation skills, as well as work in team skills, have been improved.

As the manual system has been analyzed many times, business analytical skills developed. Programming skills, as well as logical thinking and problem-solving skills, have positively affected other areas of life. Finally, writing the dissertation helped to discover how to compose a technical document in academic terms.

References

- [1] HQ Rental Software, "Compare Our Plans," 2022. [Online]. Available: <https://hqrentalsoftware.com/pricing/>. [Accessed 02 March 2022].
- [2] Software Advice, "About RentSyst," 2022. [Online]. Available: <https://www.softwareadvice.com/crm/rentsyst-profile/>. [Accessed 02 March 2022].
- [3] M. Martin, "What is a Functional Requirement in Software Engineering? Specification, Types, Examples," 2022. [Online]. Available: <https://www.guru99.com/functional-requirement-specification-example.html>. [Accessed 03 March 2022].
- [4] M. Martin, "What is Non-Functional Requirement in Software Engineering? Types and Examples," 2022. [Online]. Available: <https://www.guru99.com/non-functional-requirement-type-example.html>. [Accessed 03 March 2022].
- [5] Tutorialspoint, "SDLC - Iterative Incremental Model," 2022. [Online]. Available: https://www.tutorialspoint.com/adaptive_software_development/sdlc_iterative_incremental_model.htm. [Accessed 24 06 2022].
- [6] Visual Paradigm, "What is Unified Modeling Language (UML)?," 2022. [Online]. Available: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/>. [Accessed 28 06 2022].
- [7] A. Walker, "UML Use Case Diagram: Tutorial with EXAMPLE," 2022. [Online]. Available: <https://www.guru99.com/use-case-diagrams-example.html>. [Accessed 05 March 2022].
- [8] A. Walker, "UML Class Diagram Tutorial: Abstract Class with Examples," 2022. [Online]. Available: <https://www.guru99.com/uml-class-diagram.html>. [Accessed 05 March 2022].
- [9] R. Peterson, "ER Diagram: Entity Relationship Diagram Model | DBMS Example," 2022. [Online]. Available: <https://www.guru99.com/er-diagram-tutorial-dbms.html>. [Accessed 06 March 2022].
- [10] Data Flair, "Pros and Cons of Java," 2022. [Online]. Available: <https://data->

- flair.training/blogs/pros-and-cons-of-java/. [Accessed 29 06 2022].
- [11] M. McLean, "8 Advantages of Using MySQL," 2017. [Online]. Available: <https://devops.com/8-advantages-using-mysql/>. [Accessed 30 06 2022].
- [12] K. Chandrakant, "Why Choose Spring as Your Java Framework?," 11 March 2022. [Online]. Available: <https://www.baeldung.com/spring-why-to-choose>. [Accessed 01 07 2022].
- [13] Java T Point, "Advantages of Hibernate Framework," 2021. [Online]. Available: <https://www.javatpoint.com/hibernate-tutorial>. [Accessed 01 07 2022].
- [14] Hibernate, "What is Object/Relational Mapping?," 2021. [Online]. Available: <https://hibernate.org/orm/what-is-an-orm/>. [Accessed 01 07 2022].
- [15] Java T Point, "What is Tomcat?," 2021. [Online]. Available: <https://www.javatpoint.com/what-is-tomcat>. [Accessed 01 07 2022].
- [16] Tutorials Point, "IntelliJ Idea - Introduction," 2022. [Online]. Available: https://www.tutorialspoint.com/intellij_idea/intellij_idea_introduction.htm. [Accessed 01 07 2022].
- [17] MySQL, "MySQL Workbench," 2022. [Online]. Available: <https://www.mysql.com/products/workbench/>. [Accessed 01 07 2022].
- [18] Visual Paradigm, "Benefits of Designing Database with Visual Paradigm," 2022. [Online]. Available: https://www.visual-paradigm.com/support/documents/vpuserguide/3563/3584/85402_benefitsofde.html#:~:text=Visual%20Paradigm%20allows%20you%20to%20reverse%20engineering%20Java%20classes%20and,the%20models%20for%20further%20development.. [Accessed 01 07 2022].
- [19] R. P. M. M. P. F. G. Paolone, "Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator," p. 18, 04 February 2021.
- [20] Java T Point, "Unit Testing," 2021. [Online]. Available: <https://www.javatpoint.com/unit-testing>. [Accessed 03 07 2022].
- [21] Java T Point, "Integration testing," 2021. [Online]. Available: <https://www.javatpoint.com/integration-testing>. [Accessed 03 07 2022].
- [22] System Testing, 2022. [Online]. Available: <https://www.geeksforgeeks.org/system-testing/>. [Accessed 03 07 2022].

- [23] Baeldung, "The Spring @Controller and @RestController Annotations," 2022. [Online]. Available: <https://www.baeldung.com/spring-controller-vs-restcontroller#:~:text=Spring%204.0%20introduced%20the%20%40RestController,class%20with%20the%20%40ResponseBody%20annotation..> [Accessed 05 07 2022].
- [24] Pankaj, 2022. [Online]. Available: <https://www.digitalocean.com/community/tutorials/spring-restcontroller>. [Accessed 04 07 2022].
- [25] M. Martin, "MVC Framework Tutorial for Beginners: What is, Architecture & Example," 2022. [Online]. Available: <https://www.guru99.com/mvc-tutorial.html>. [Accessed 06 07 2022].

Appendix A – System Manual

A.1 Hardware requirements

Hardware	Recommended Requirements
Processor	Intel core i3 or better
Memory	4 GB RAM or more
Free storage	4 GB or more
Printer	Ink jet or suitable printer
Internet	Minimum 512Kpbs or 1Mbps or higher

Table A.1: System Hardware Requirements.

A.2 Software requirements

Software	Recommended Requirements
Operating system	Microsoft Windows 7 or newer
Java	Version 11
MySQL	Version 8.0
Web Server	Apache Tomcat 8.5
Web browser	Google chrome or Firefox or MS Edge
Code editor	IntelliJ IDEA
MySQL Workbench	Version 8.0

Table A.2: System Software Requirements.

A.3 Installation & Configuration

A.3.1 Install Java Runtime Environment

The following steps can be followed in order to install the Java Runtime Environment. (JRE)

Step 1: Run the installation wizard and click next.



Figure A.1: Java Runtime Environment Installation-Welcome Page.

Step 2: Select the destination folder and click next.

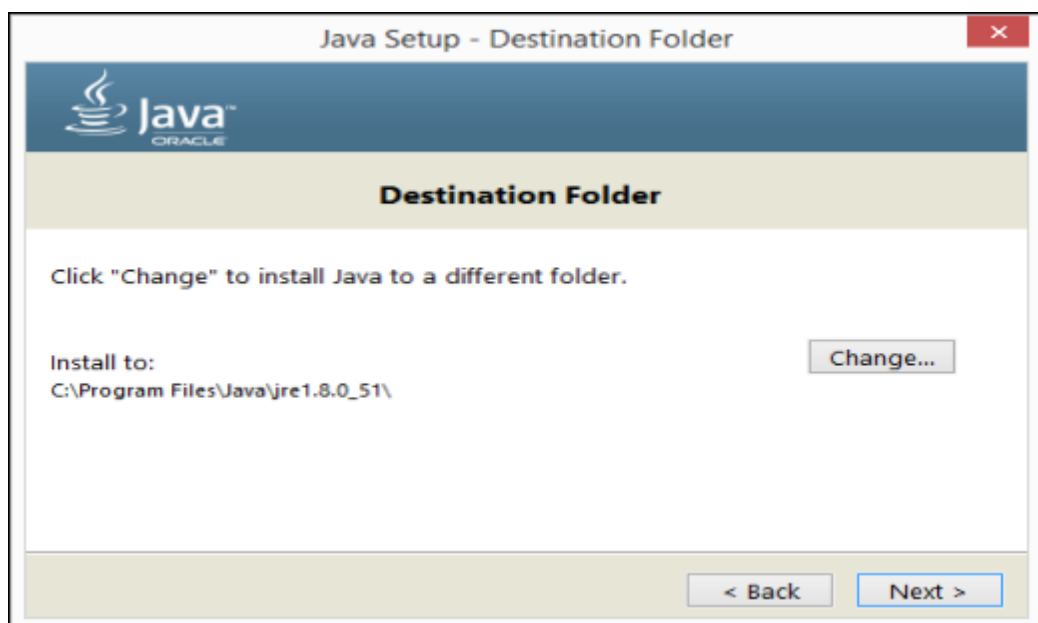


Figure A.2: Java Runtime Environment Installation-Destination Folder.

Step 3: After installation, click Finish to close the setup.



Figure A.3: Java Runtime Environment Installation-Finish.

A.3.2 Installation of MySQL Server

Step 1:

After accepting the End User License Agreement, you will be prompted with the selection of installation type. Choose 'Custom' as shown below.

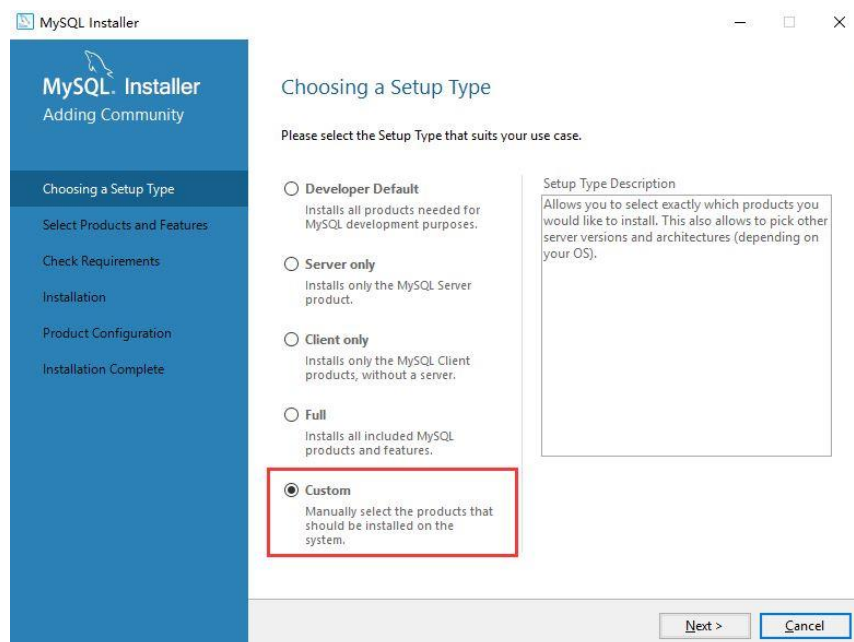


Figure A.4: MySQL Server Installation-Setup Type.

Step 2: Click next to bring up the screen to choose products for installation.

The following figure shows installable products and features on the MySQL server installation wizard.

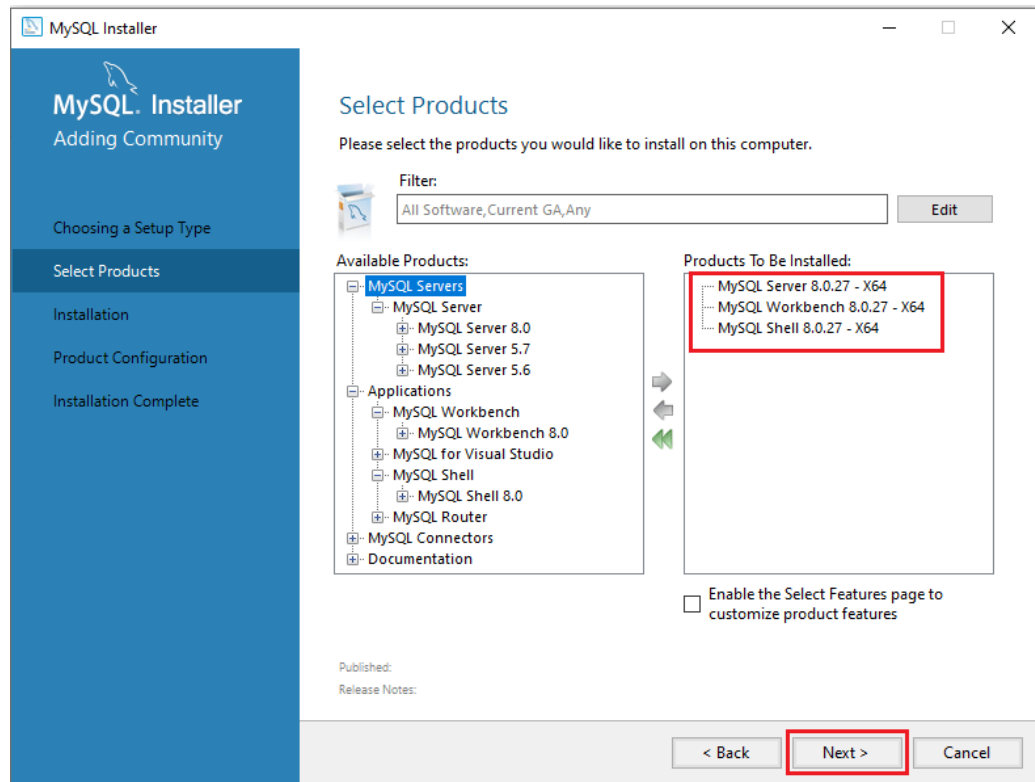


Figure A.5: MySQL Server Installation-Select Products.

Step 3: Expand MySQL Server and choose MySQL Server 8.0

Step 4: Go to Applications -> MySQL Workbench 8.0 and choose workbench.

Step 5: Click on Documentation and select MySQL Documentation.

Step 6: Expand Documentation, if not already expanded, and choose Sample and Example 8.0.

Step 7: Click Next. Four items should be ready for installation – server, workbench, documentation, and samples. Click Execute.

Step 8: After installation is completed, click finish.

Appendix B – Design Documentation

B.1 Entity Relationship (ER) Diagrams

B.1.1 ER Diagram of Vehicle Management Module

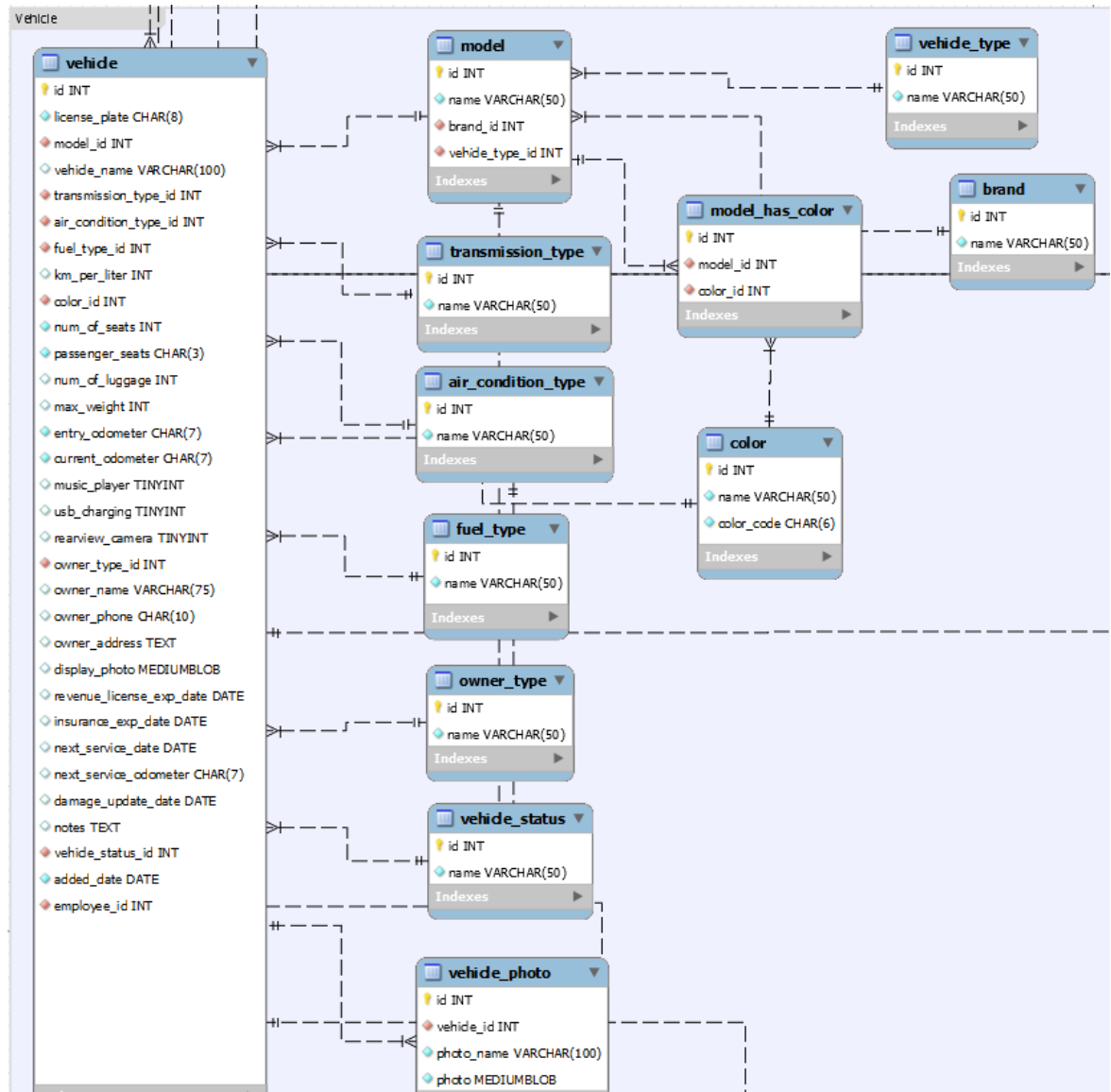


Figure B.1: ER Diagram of Vehicle Management Module.

B.1.2 ER Diagram of Package Management Module

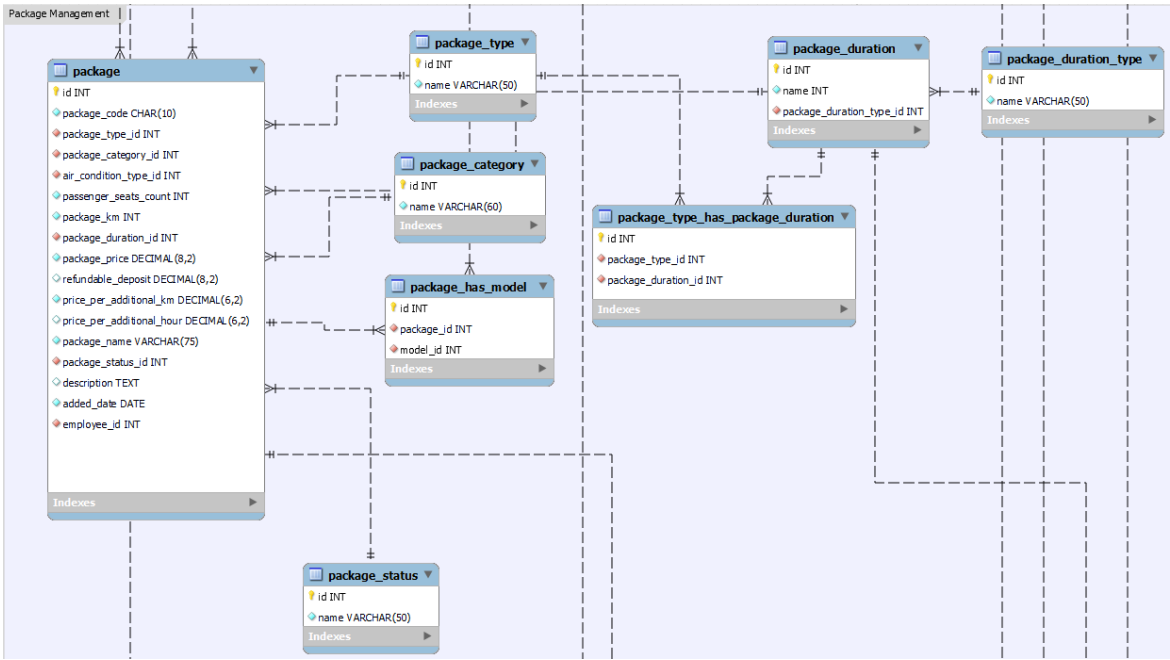


Figure B.2: ER Diagram of Package Management Module.

B.1.3 ER Diagram of Chauffeur-Drive Reservation Management Module

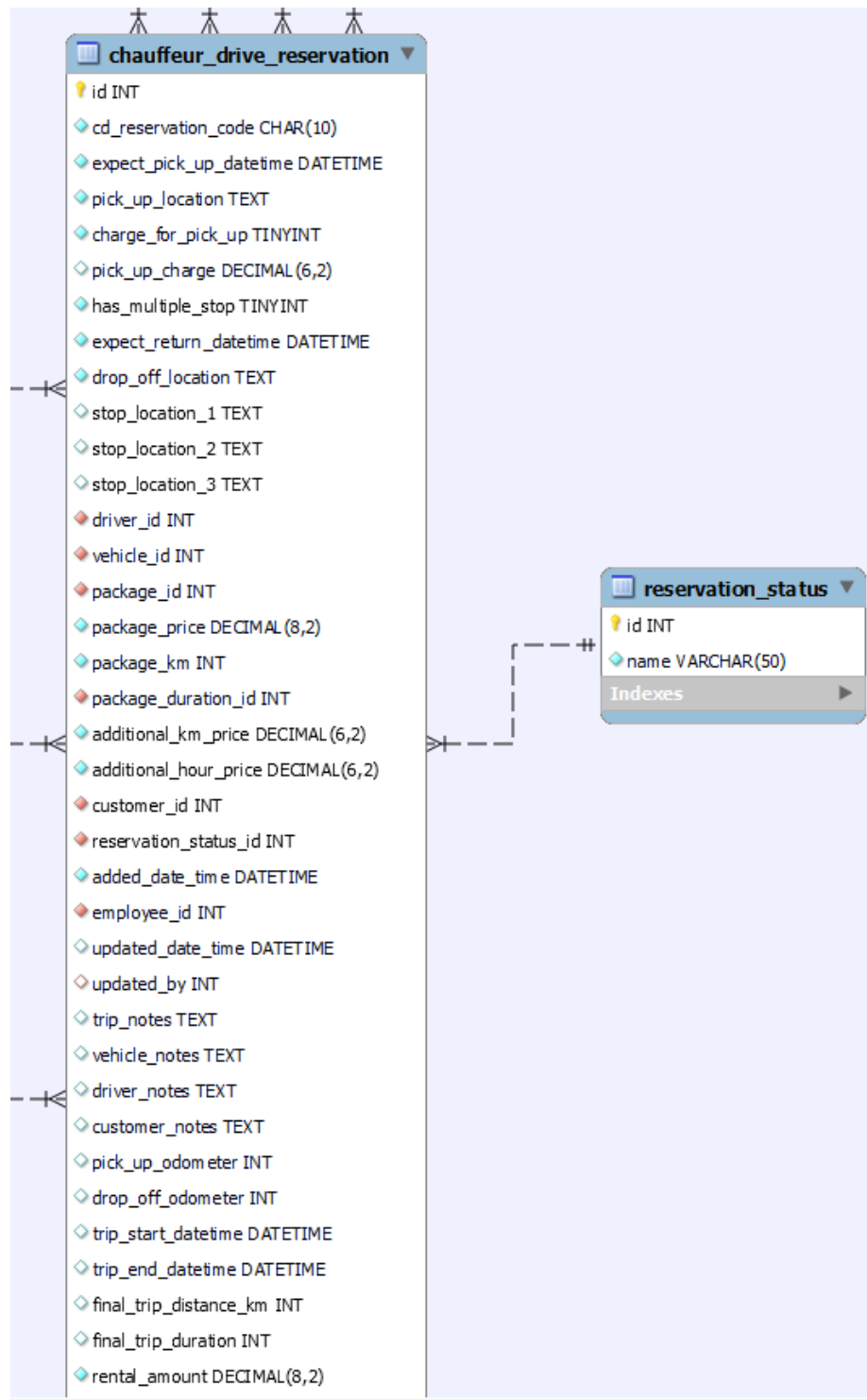


Figure B.3: ER Diagram of Chauffeur-Drive Reservation Management Module.

B.1.4 ER Diagram of Customer Management Module

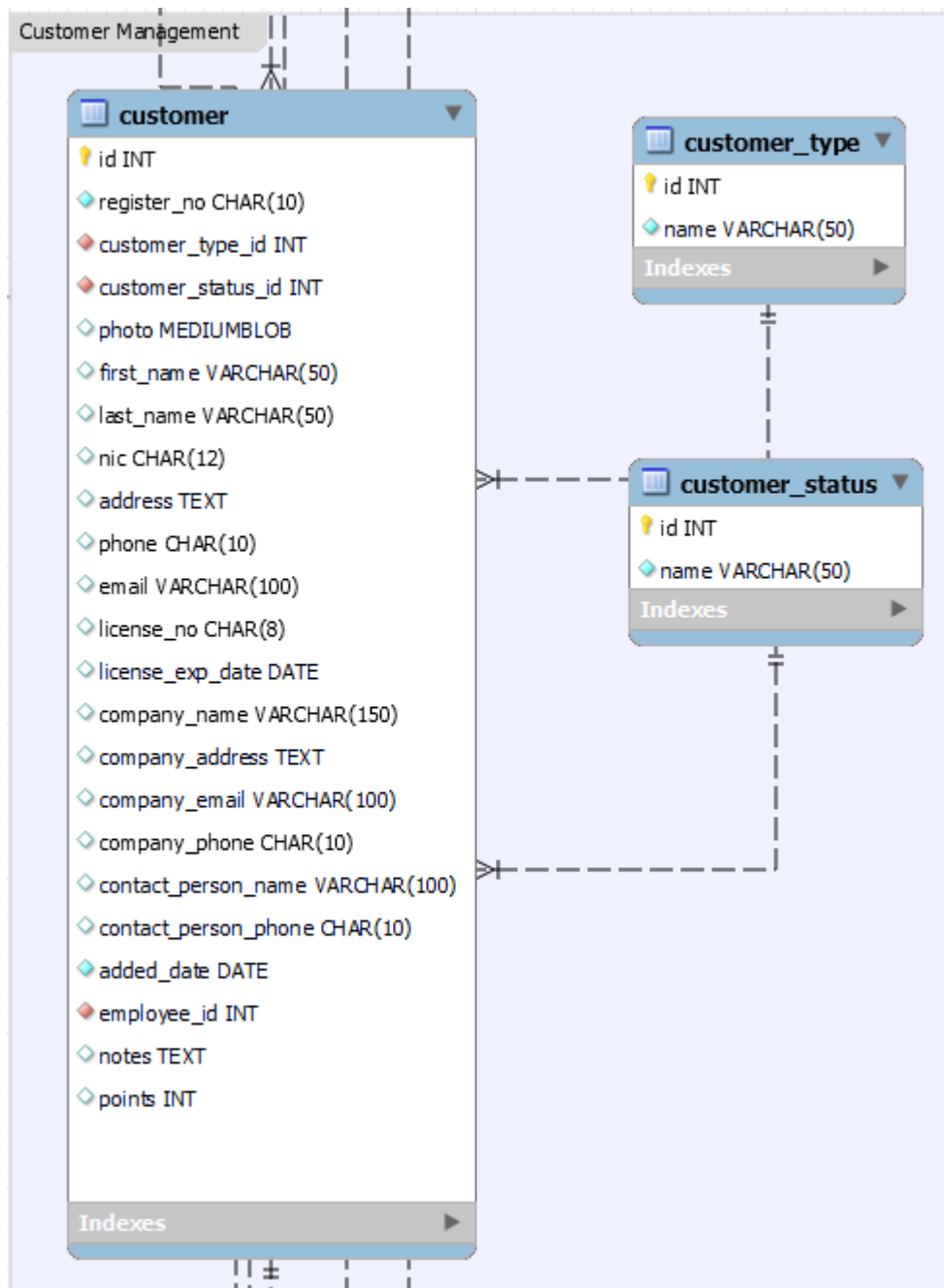


Figure B.4: ER Diagram of Customer Management Module.

B.2 Use Case Diagrams & Narratives

B.2.1 Use case diagram for Manager's Activities

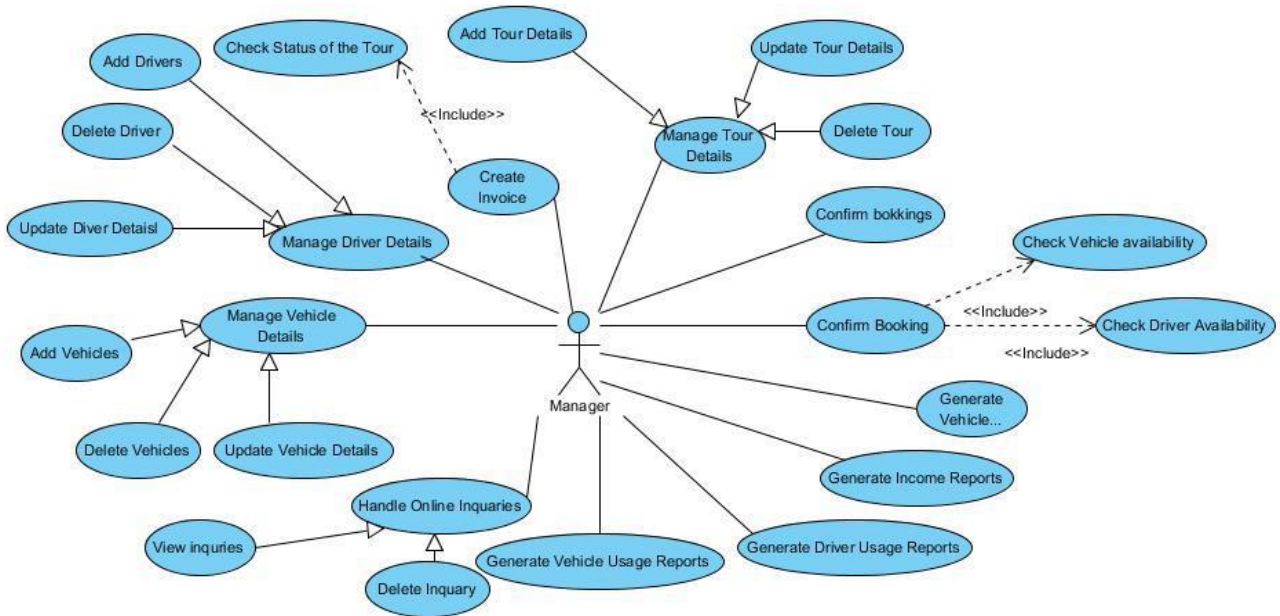


Figure B.5: Use Case Diagram of Manager's Activities.

B.2.2 Use narrative diagram for Manager's Activities

Use Case	Manager activities
Actors	Manager
Description	Manager can add, update and delete vehicles, tours, drivers and confirm bookings. Manager can generate reports such as revenue reports.
Pre-Conditions	Manager must have logged into the system.
Flow of events	1. Manager logs into the system. 2. Manager add, update and delete vehicles, drivers or tours, and generate reports.

Table B.1: User Case Narrative for Manager's Activities

B.3 Sequence Diagram

B.3.1 Sequence Diagram for Login to the System

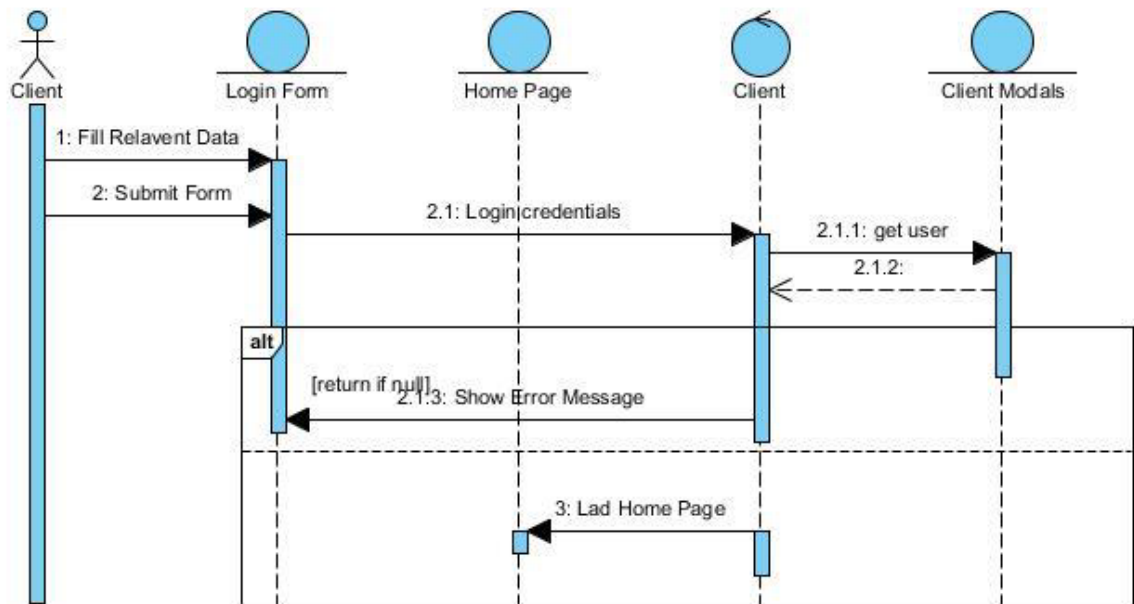


Figure B.6: Sequence Diagram of the System Login.

Appendix C – User Manual

By referring to the user manual, the user will understand system components and navigate easily through the system. Different user levels documents are available to other system users. However, the user manual demonstrated here is based on the manager's account.

C.1 Main Window Related Components

The main window (dashboard) contains the sidebar to the left and the nav bar to the top. This section is dedicated to demonstrating more details on these components.

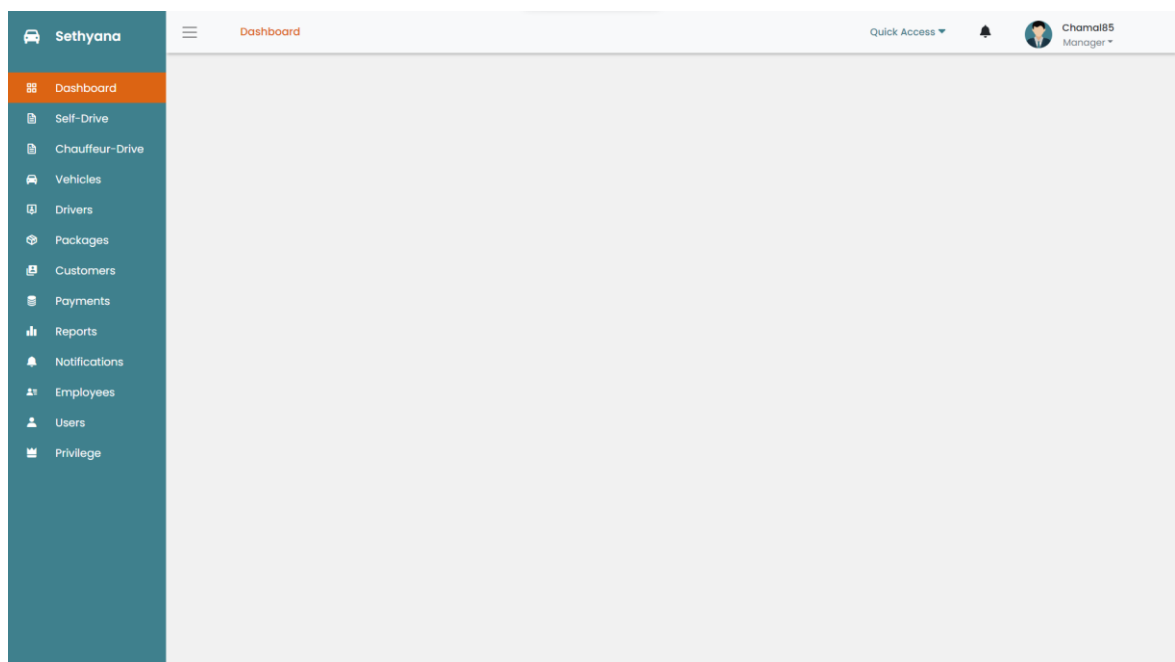


Figure C.1: Dashboard of the System.

C.1.1 Sidebar

The Sidebar is the main component to navigate through the different modules. The mini sidebar can be used on small screens devices switching the sidebar toggler.

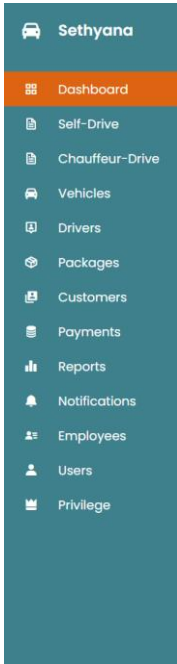


Figure C.2: Sidebar of the System.

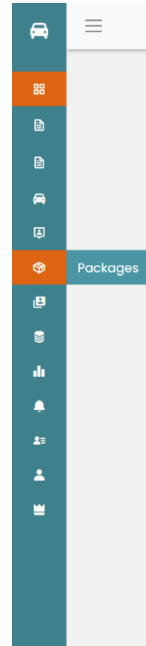


Figure C.3: Mini Sidebar of the System.

C.1.2 Nav Bar

Navbar contains the Side Bar Toggler, Breadcrumb, Quick Access, Notification Bell, and the Logged User section from left to right respectively.

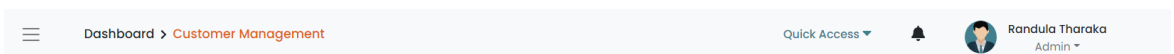


Figure C.4: Nav Bar of the System.

Breadcrumb

Breadcrumb indicates the location that the user is currently in and can easily be navigated backward all the way to the dashboard.

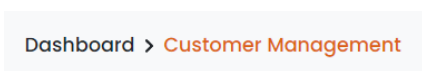


Figure C.5: Breadcrumb of the system.

Quick Access

Quick Access provides direct links to perform tasks such as creating new reservations, checking the availability of vehicles and drivers, and adding new customers and vehicles to the system.

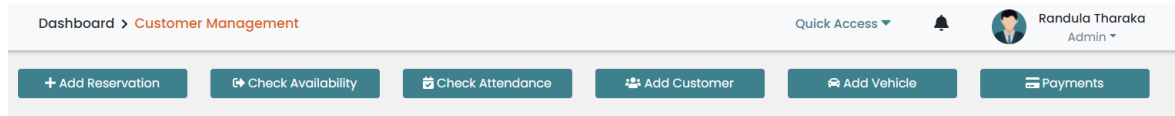


Figure C.6: Quick Access of the System.

Notifications Bell

Notification Bell shows important messages related to events such as vehicle maintenance due date and driver license expiration date.

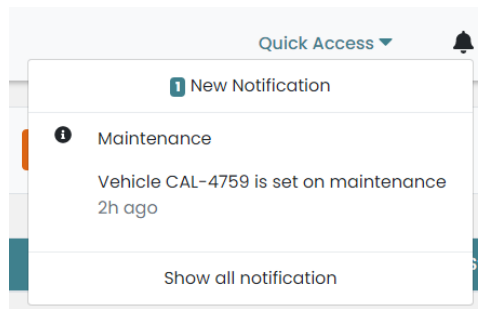


Figure C.7: Notifications of the System.

Logged User Profile

This section of the nav bar displays the user name and user role of the currently logged user. Clicking on the user name displays a pop-up that facilitates functions such as viewing the user profile, changing the password, and user signing out from the system.

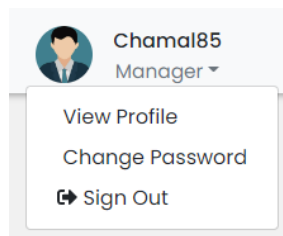
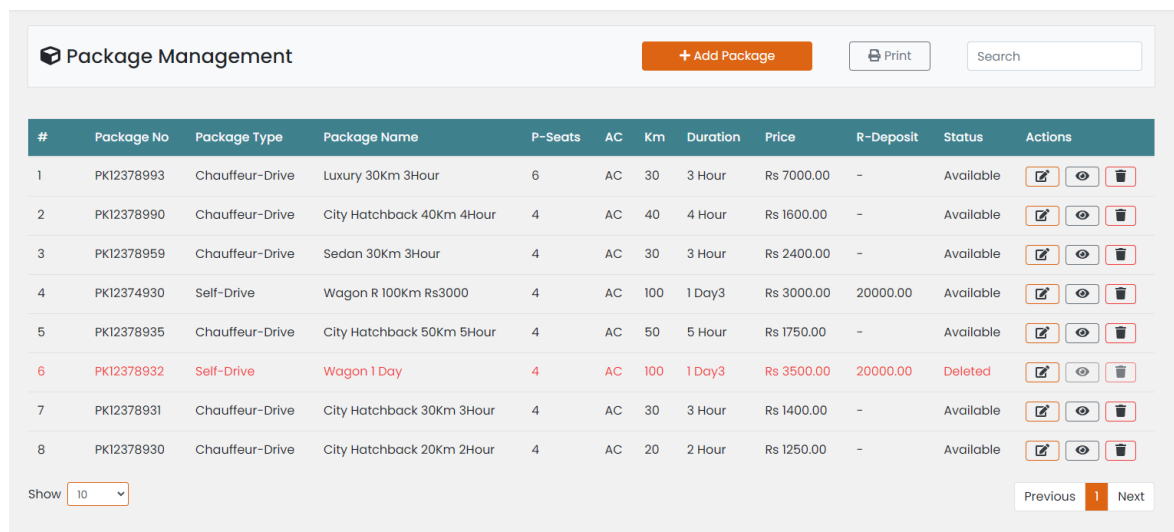


Figure C.8: Logged User Profile.

C.2 Table Related Components

Most modules in the system cooperated with many numbers for records. The implemented table design allows these records to be easily manipulated to the user's requirement.



#	Package No	Package Type	Package Name	P-Seats	AC	Km	Duration	Price	R-Deposit	Status	Actions
1	PK12378993	Chauffeur-Drive	Luxury 30Km 3Hour	6	AC	30	3 Hour	Rs 7000.00	-	Available	  
2	PK12378990	Chauffeur-Drive	City Hatchback 40Km 4Hour	4	AC	40	4 Hour	Rs 1600.00	-	Available	  
3	PK12378959	Chauffeur-Drive	Sedan 30Km 3Hour	4	AC	30	3 Hour	Rs 2400.00	-	Available	  
4	PK12374930	Self-Drive	Wagon R 100Km Rs3000	4	AC	100	1 Day3	Rs 3000.00	20000.00	Available	  
5	PK12378935	Chauffeur-Drive	City Hatchback 50Km 5Hour	4	AC	50	5 Hour	Rs 1750.00	-	Available	  
6	PK12378932	Self-Drive	Wagon 1 Day	4	AC	100	1 Day3	Rs 3500.00	20000.00	Deleted	  
7	PK12378931	Chauffeur-Drive	City Hatchback 30Km 3Hour	4	AC	30	3 Hour	Rs 1400.00	-	Available	  
8	PK12378930	Chauffeur-Drive	City Hatchback 20Km 2Hour	4	AC	20	2 Hour	Rs 1250.00	-	Available	  

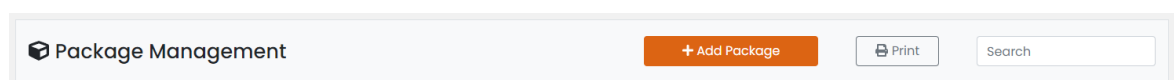
Show

Previous **1** Next

Figure C.9: Table Related Components.

C.2.1 Table Header

The table heading bar contains the module name, add record button, print button, and the table search bar from left to right respectively, as shown in the figure.



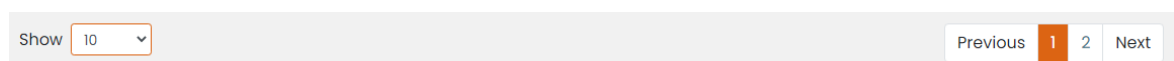
Package Management	+ Add Package	Print	Search
--------------------	---------------	-------	--------

Figure C.10: Table Header

- Add Button: Opens the form for the relevant module.
- Print Button: Prints all the records shown in the table.
- Search Bar: Searches a specific record to given criteria.

C.2.2 Pagination

Users can select how many records need to be displayed on the table by altering values in the 'Show' combo box. When a module has more records than the current table displays, the user can use the pagination bar to navigate to different pages of the table.



Show 10	Previous	1	2	Next
--------------------------------------	----------	----------	---	------

Figure C.11: Table Pagination.

C.3 User Login

Users can log in to the system by providing the correct user name and password and clicking the 'Login' Button. When a user enters an incorrect username and/ or password, the system displays an error message to re-enter the correct credentials.

In case the user forgets their password, the system is facilitated to reset the password after confirming the user's email address. To do that, click on 'Forgot Password?' and follow the wizard.

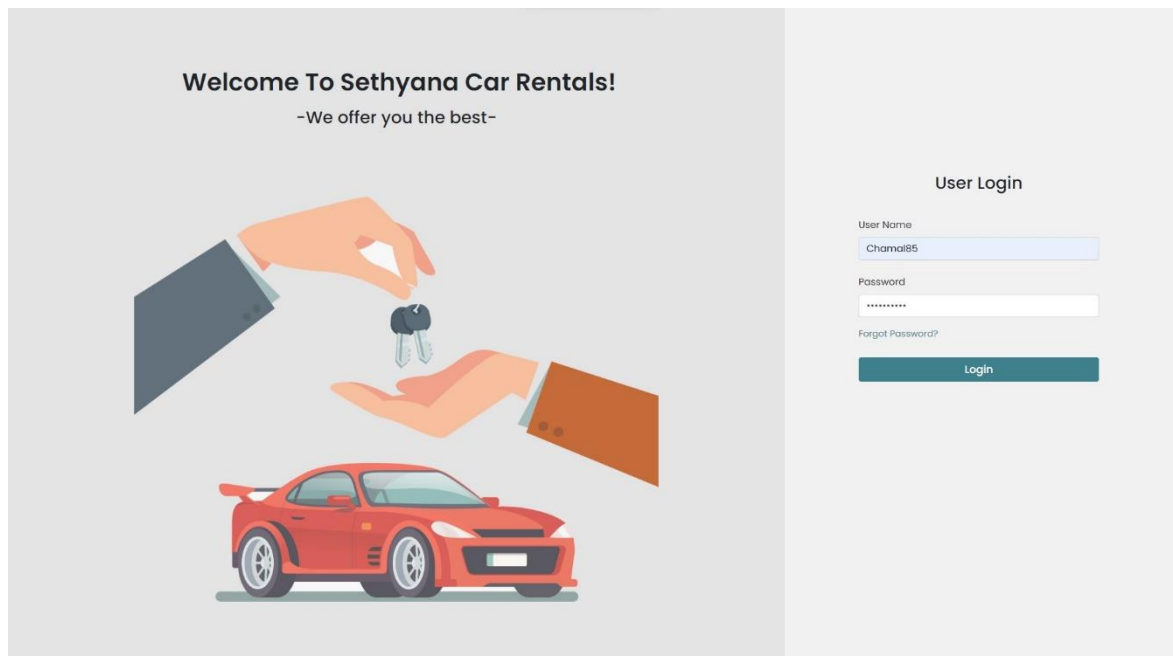


Figure C.12: User Login of the System.

C.4 Creating a New Chauffeur-Drive Reservation

Making a new reservation is guided through a tab design wizard. Users can navigate through each tab from left to right and fill in the required details to create a new reservation. A Field label marked with a red asterisk (*) denotes that it is a required field, and the user cannot leave it blank.

Follow each tab wizard and the instructions mentioned below to create a new chauffeur-drive reservation.

C.4.1 Time & Location Tab

Here the user can fill time and location related details regarding the new reservation. If the customer's pick up location is far from the office, the user can specify the pick up charge by selecting the 'Yes' radio button under the 'Charge for pick up?'. The system is facilitated to add a few multiple stops before the drop off location by selecting the relevant radio button. The 'Get Duration' option calculates the duration of pick-up and drop-off time. User can open the google map by clicking the 'Open Map.'

The screenshot shows the 'Add Reservation' form with the 'Time & Location' tab selected. The form is divided into several sections with red asterisks indicating required fields. The 'Pick Up Date & Time' field is set to '06/19/2022 12:10 AM'. The 'Pick Up Location' field is set to 'Kaduwele'. The 'Drop Off Date & Time' field is set to '06/19/2022 02:10 AM'. The 'Drop Off Location' field is set to 'Punchi Borella'. There are two radio button options: 'Charge for pick up?' with 'No' selected, and 'Have multiple stops?' with 'No' selected. At the bottom, there are two buttons: 'Get Duration' and 'Open Map'. The form is part of a larger dashboard with a sidebar on the left containing links to 'Dashboard', 'Self-Drive', 'Chauffeur-Drive', 'Vehicles', 'Drivers', 'Packages', 'Customers', 'Payments', 'Reports', 'Notifications', 'Employees', 'Users', and 'Privilege'. The top right corner shows the user's name 'Chamal85 Manager' and a 'Quick Access' dropdown.

Figure C.13: Chauffeur-Drive Reservation-Time & Location Tab.

C.4.2 Package Tab

In the package tab, the required duration and distance will be displayed on the top. User can use the filter section to filter out the available packages according to customer needs. Then, click on the relevant radio button to select the required package.

Sethyana Dashboard > Chauffeur-Drive Reservations Quick Access Chamal85 Manager

Add Reservation

Time & Location Package Vehicle Driver Customer Confirm

Required Distance : 47 Km Required Duration : 2 Hours

Filters:

Package Category: Nothing selected Vehicle Model: Toyota Vitz 2018 Package Kilometers: Nothing selected Package Duration: Nothing selected

Available Packages

#	Package Category	Package Name	Passenger Seats	AC	KM	Duration	Add. Km Price	Add. Hour Price	Package Price
☒ 1	City Hatchback	City Hatchback 2Hour 20Km	4 Seats	AC	20 Km	2 Hour	Rs 47.00	Rs 240.00	Rs 1250.00
☐ 2	City Hatchback	City Hatchback 3Hour 30Km	4 Seats	AC	30 Km	3 Hour	Rs 47.00	Rs 240.00	Rs 1400.00
☐ 3	City Hatchback	City Hatchback 4Hour 40Km	4 Seats	AC	40 Km	4 Hour	Rs 47.00	Rs 244.00	Rs 1600.00
☐ 4	City Hatchback	City Hatchback 5Hour 50Km	4 Seats	AC	50 Km	5 Hour	Rs 47.00	Rs 240.00	Rs 1750.00
☐ 5	City Hatchback	City Hatchback 100Km Rs6000	4 Seats	AC	100 Km	10 Hour	Rs 47.00	Rs 240.00	Rs 6000.00

Show 10 Previous 1 2 Next

Figure C.14: Chauffeur-Drive Reservation-Package Tab.

C.4.3 Vehicle Tab

All the available vehicles will be displayed here alongside vehicle details according to the previously selected vehicle model and pick up and drop off dates. Click on the relevant radio button to select the required vehicle.

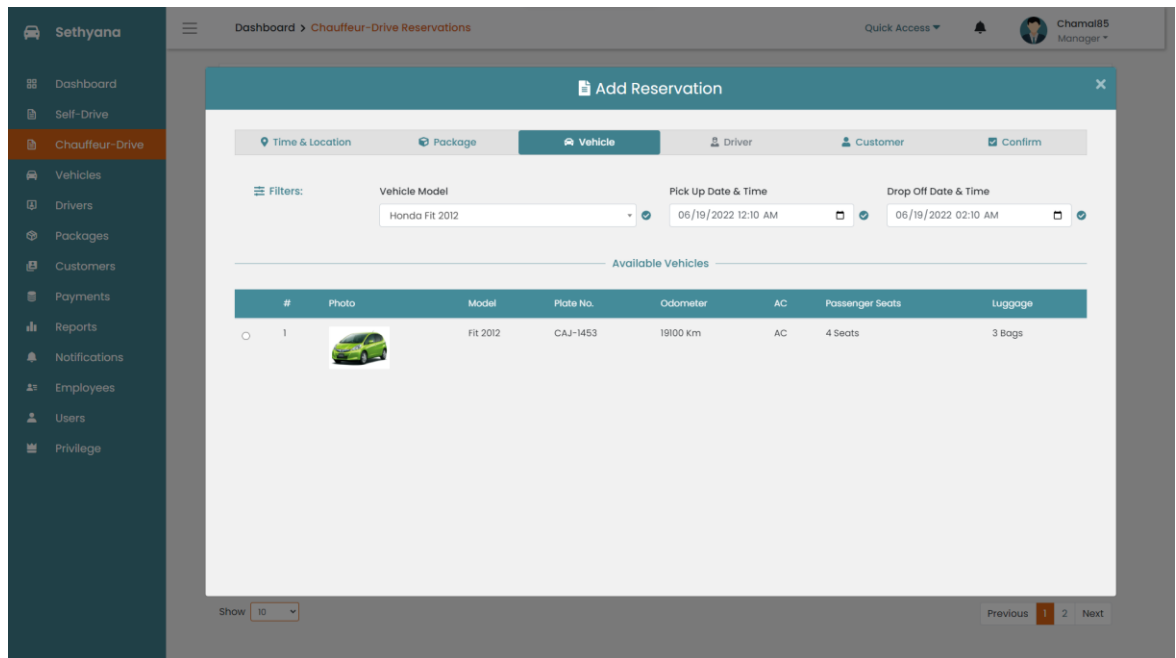


Figure C.15: Chauffeur-Drive Reservation-Vehicle Tab.

C.4.4 Driver Tab

In the driver tab, all the available drivers will be shown according to the specified pick up and drop off times and dates. The user can search for a particular driver by driver calling name or license number. Click on the relevant radio button to select the preferred driver.

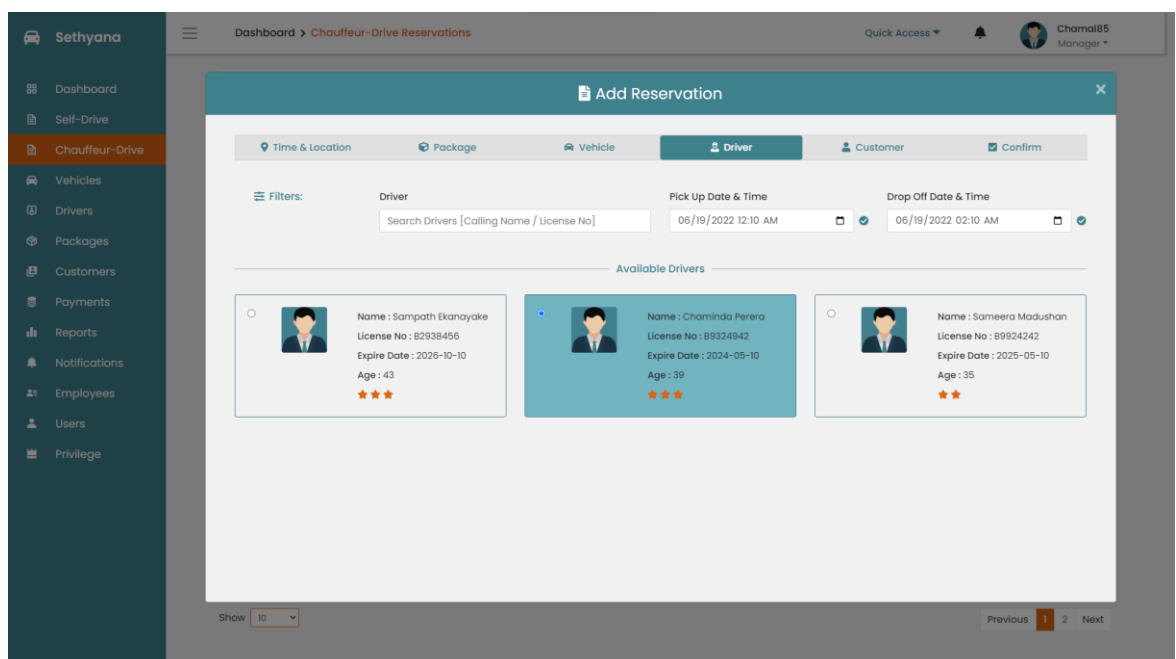


Figure C.16: Chauffeur-Drive Reservation-Driver Tab.

C.4.4 Customer Tab

User can search and select the customer if the customer exists in the database. The system is also facilitated to update customer information right on this tab without navigating elsewhere. User can also add a new customer by clicking the link ‘New Customer’ and filling in the correct information.

The screenshot shows a web application interface for 'Sethyana'. The main navigation menu on the left includes: Dashboard, Self-Drive, Chauffeur-Drive (selected), Vehicles, Drivers, Packages, Customers, Payments, Reports, Notifications, Employees, Users, and Privilege. The top navigation bar shows 'Dashboard > Chauffeur-Drive Reservations' and a user profile for 'Chamal85 Manager'. The main content area is titled 'Add Reservation' and has tabs for 'Time & Location', 'Package', 'Vehicle', 'Driver', 'Customer' (selected), and 'Confirm'. The 'Customer' tab contains a form with the following fields: 'Customer Type' (dropdown menu set to 'Individual'), 'Customer' (text field with 'Rukshan Ruchiraa (License: 962452348V)' and a 'New Customer' link), 'First Name' (text field with 'Rukshan'), 'Last Name' (text field with 'Ruchiraa'), 'Phone' (text field with '0778719760'), 'Email' (text field with 'rukshan@gmail.com'), 'NIC' (text field with '962452348V'), and 'Address' (text field with '2/A, 2nd Lane, Samagi Mawatha'). A red asterisk indicates required fields. At the bottom, there is a 'Show' dropdown set to '10' and pagination links for 'Previous', '1', '2', and 'Next'.

Figure C.17: Chauffeur-Drive Reservation-Customer Tab.

C.4.4 Confirm Tab

A summary of the reservation can be seen on the left side of the confirmation tab. User can add a note to the reservation, and the driver will also be able to see it. By default, reservation status is selected as ‘reserved’; however: if the reservation is not confirmed, the user can select ‘booking’ from the drop-down. Finally, the user can click on the ‘Add Reservation’ button to add the reservation.

Sethyana

Dashboard

Self-Drive

Chauffeur-Drive

Vehicles

Drivers

Packages

Customers

Payments

Reports

Notifications

Employees

Users

Privilege

Dashboard > Chauffeur-Drive Reservations

Quick Access

Chamal85 Manager

Add Reservation

Time & Location

Package

Vehicle

Driver

Customer

Confirm

Summary

Notes

Reservation# CDR0000016

Customer : Rukshan Ruchira

NIC : 962452348V

Driver Name : Chaminda Perera

Driving License : B9324942

Vehicle Model : Honda Fit 2012

License Plate : CAJ-1453

Pick Up Date : 2022-06-19 12:10 AM

Drop Off Date : 2022-06-19 02:10 AM

Pick Up Location : Kaduwela

Drop Off Location : Punchi Borella

Package Km : 20

Package Duration : 2 Hour

Total Price : 1250

Notes about reservation

Reservation Status *

Select Reservation Status

Update Reservation

Add Reservation

Added By : Niluni Wathsala

Added Date : 2022-06-18 05:40 AM

Updated By : Chamal Wijerathna

Updated Date : 2022-06-26 04:23 AM

Show 10

Previous 1 2 Next

Figure C.18: Chauffeur-Drive Reservation-Confirm Tab.

C.4.5 Printing Payment Receipt

Following Figure C.19 shows a system-generated payment receipt for a chauffeur-driven tour.

Sethyana Car Rental	
141/1, RATHNAPURA ROAD, AVISSAWELLA.	
Tel : 075-7-611-611 / 075-5-570-570	
Date: 2022-06-26 04:36 AM	

Invoice No.	PAY0000020
Reservation No.	CDR0000018
Customer	Rukshan Ruchiraa
Driver	Chaminda Perera
Pick Up	Kaduwela
Drop Off	Borella
Trip Distance	100 Km
Trip Duration	

Package Price	Rs 1400.00
Addt. Time Charge	
Addt. Km Charge	Rs 3290.00 (70 Km X Rs 47.00)
Pick Up Charge	Rs 200.00

Total	Rs 4890.00
Cash	5000.00
Change	110.00

Thank You!	

Figure C.19: Generated Customer Payment Receipt.

Appendix D – Management Reports

D.1 Chauffeur-Drive Rental Reports

This report type produces daily chauffeur-drive rental count as well as daily revenue for a specified time duration. User can specify the start and end date from the date fields and click ‘Generate Report.’ The bar chart will be shown, and the details can be seen in the table.

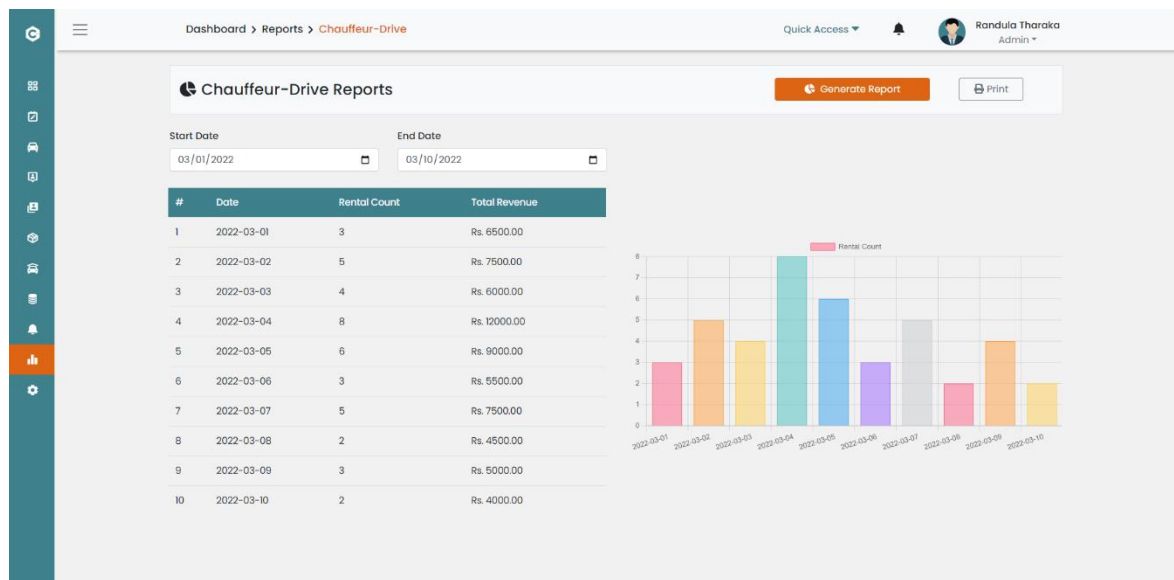


Figure D.1: Chauffeur-Drive Rental Report.

D.2 Revenue Reports

The system is facilitated to generate revenue reports based on day, week, or month. User can select a start and end date from the date picker. Then select ‘Report Type’ and click ‘Generate Report’ in order to retrieve the revenue report.

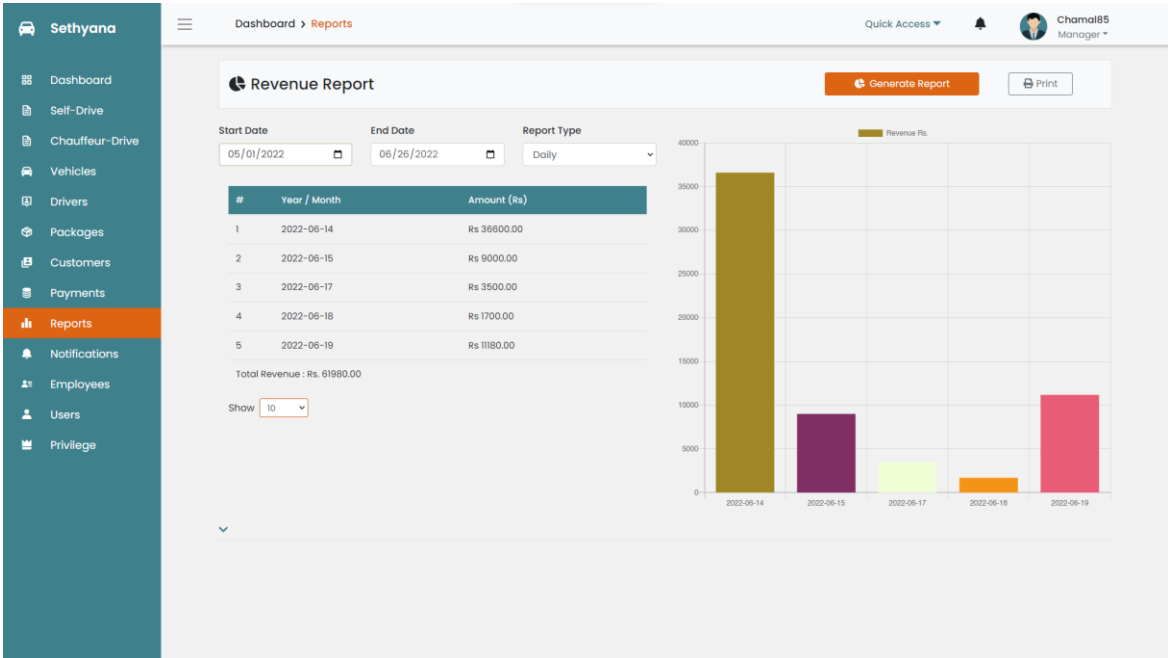
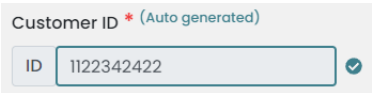
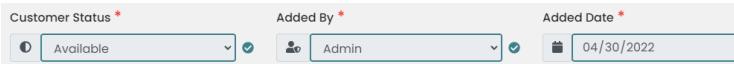
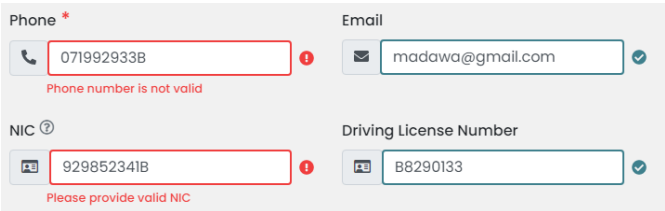
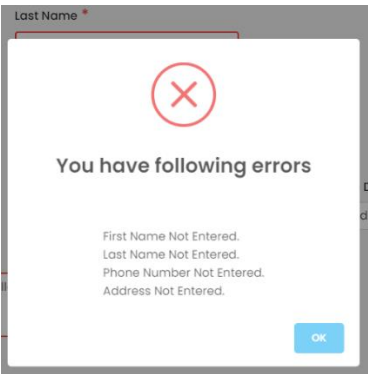


Figure D.2: Revenue Report.

Appendix E – Test Results

E.1 Test Results for Customer Module

Test No.	Test Description	Test Result	Status
	Customer ID autogenerated on customer form load.		Pass
	Default values on customer form load.		Pass
	Data validation.		Pass
	Add customer while some required fields are left blank. (negative)		Pass
	Add customer (positive)		Pass

		 Are you sure to add following customer...? Customer ID : 1122342425 First Name : Nuwan Last Name : Perera Address : No: 35/B, Athurugiriya. Phone : 0719928292 Email : nuwan88p@gmail.com NIC : 883924703V customer Status : Available Added Date : 2022-04-30 Cancel OK	
		 Your work has been Done Save SuccessFully..!	
	On update, Status enabled & Add button becomes disabled.	Customer Status * Available Added By * Admin Clear Update + Add Customer Type * Individual	Pass
	Display user confirmation on update & 'update success' messages.	 Are you sure to update following customer details...? Phone Number is Changed: 0719399393 --> 0719399392 NIC is Changed: 939939990V --> 939939991V Cancel Update	Pass

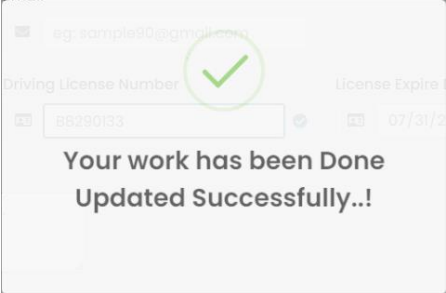
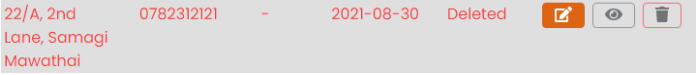
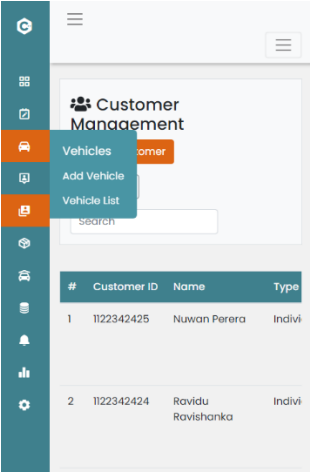
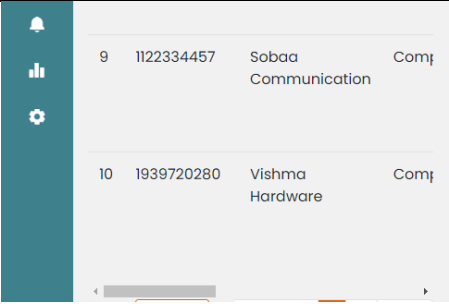
			
--	--	--	--

Table E.1: Test Results for Customer Module.

E.2 Test Results for Common Test Cases

Test No.	Test Description	Test Result	Status
	View & Delete buttons are disabled on a deleted record.		Pass
	Collapsible Quick Access bar appears on click.		Pass
	Display 'mini side bar' on small screens.		Pass
	Display Horizontal Ruler on small screens.		Pass
	Help tooltip appears on mouse hover on '?' icon.		Pass

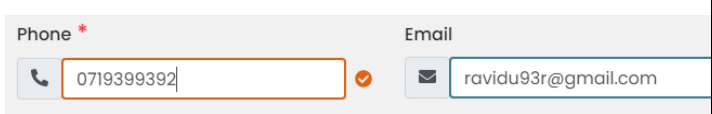
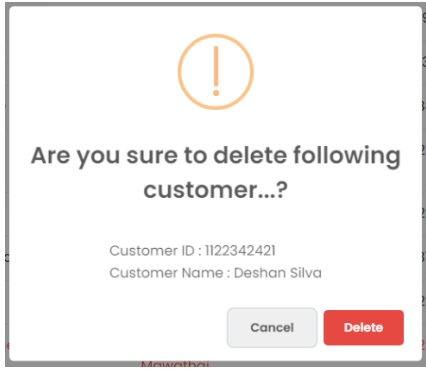
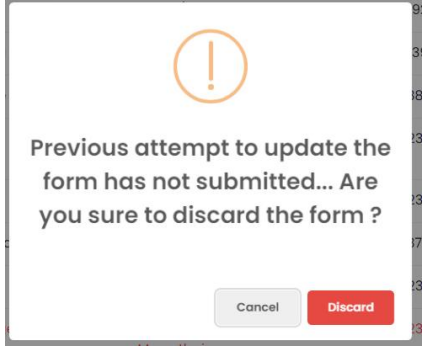
			
	On update, change border color of updating field.		Pass
	Display user confirmation message for record delete on a table.		Pass
	Display discard confirmation message trying to update a new record while another one's details are loaded on the form.		Pass

Table E.2: Test Results of Common Test Cases.

Appendix F – Client Certificate



Phone: 075 5 611 611 / 075 5 570 570

Email: sethyanarentacar@gmail.com

Location: 141/1, Rathnapura Road, Avissawella.

26th April 2022

Project Examination Board,
University of Colombo of School Computing,
UCSC Building Complex,
35, Reid Avenue,
Colombo 07.

Dear Sir/ Madam,

Letter of Certification

This is to certify that Mr. M.V.G.R. Tharaka has successfully completed the Vehicle Rental & Reservation Management System for our company Sethyana Rent a Car and Cab Service as his final year project for the BIT degree.

He has previously gathered data and information from our premises and I am pleased to inform that the developed system has fulfilled the business requirements of our company and increased the efficiency and the productivity of our business activities.

I would like to appreciate Mr. Tharaka for his time and effort in developing this system for our company.

Thank you.

Yours Faithfully,



A.P. Chamal Wijerathna

Manager

Sethyana Rent a Car & Cab Service

SETHYANA RENT A CAR
& CAB SERVICE
Avissawella.
Tel: 075 7 611 611
075 5 570 570

Figure F.1: Client Certificate.

Glossary

Backup – means keeping a copy of information as an external storage where usually keep in another central computer or in external disks.

Database - is an organized collection of data for one or more purposes, usually in digital form.

Functional requirements -Actual or anticipated questions which may be posed to an information retrieval system.

Graphical User Interface - is a type of user interface that allows users to interact with electronic devices with images rather than text commands.

Non-functional requirements - Criteria that can be used to judge the operation of a system, rather than specific behaviors.

Object-relational impedance mismatch – The incompatibility between object models and relational models.

Object-relational mapping – A programming technique for converting data between incompatible type systems in object-oriented programming languages.

Relational database management system - database management system (DBMS) that is based on the relational model (that all data is represented in terms of tuples, grouped into relations.)

UML – refers to Unified Modeling Language is a standardized general-purpose modeling language in the field of object-oriented engineering. This includes a set of graphic notation techniques to create visual models of object-oriented software intensive systems.

Index

A

Acceptance Testing55
Activity Diagram24
Apache..... iv, 36, 64

B

Booking..... 12, 72, 82
Browser 14, 15, 55, 64
Business.....iv, 2, 6, 9, 11, 16, 59, 60

C

Class Diagram 20, 21
CSS.....xiv, 33, 43, 47
Customeriv, 2, 3, 4, 9, 12

D

Database.....iv, 13, 14, 20, 34, 35, 36, 37, 38
Design.....iv, 5, 6, 14, 16, 17, 20, 25, 26, 27
Documentation 11, 24, 68
Driveriv, 1, 2, 3, 4, 9, 12, 13, 81

E

Encrypt 14, 34
ER Diagram 20, 22, 68, 69, 70, 71
Evaluation..... 5, 48, 55, 57, 58, 59

F

Feedback.....11, 15
Framework..... iv, 33, 35, 37
Fuel4, 8, 12
Functional Requirement.....6, 11

H

Hardware14, 33, 64
Hibernate..... iv, 35
HTML33, 43
HTTP.....xiv, 40

I

Implementation..... 5, 33, 36, 39, 40, 42, 43
IntelliJ IDEA36, 64
Interface 14, 25, 26, 27, 28, 29, 30, 31
Iterative and Incremental Modeliv, 15

J

Java iv, 33, 34, 64
JavaScript.....33, 43
jQuery 43

L

Login25, 51, 73, 78

M

Manual System..... iv, 4, 9, 10, 59
Motivation..... 1
MVC..... iv, xiv, 33, 37
MySQL iv, 34, 36

N

Non-Functional Requirements 6, 13
Notification..... 75, 76

O

Object-Oriented 17, 20, 34, 35
Observation 11, 60
Operating System 64

P

Packages 3, 4, 7, 9, 12, 13, 52, 80
Payment iv, 2, 4, 9, 12, 60, 84
Privileges 3, 11, 12, 14, 41, 55
Process Model 15, 33

Q

Quality 13, 17, 20
Query.....xiv, 35, 36, 42

R

Reliability..... 14, 49

Report..... iv, 2, 3, 4, 13, 31, 59, 85, 86
Requirement Analysis..... 6, 11, 16
Reservations iv, 1, 2, 3, 9, 12, 26, 32

S

Scope 3
Security..... iv, 13, 17, 34, 38, 41
Sequence Diagram..... 23, 73
Similar Systems..... 6
 HQ Car Rental..... 6, 7
 RentSyst..... 7, 8
Spring..... iv, 35

T

Test Cases 49, 50, 51, 52, 53, 89
Transport iv, 1

U

UML iv, 18, 37
Unit Testing 36, 48, 49
Use Case Diagram..... 9, 18, 72

V

Validation..... 1, 7, 28, 38, 47, 48, 50, 51, 52

W

Web iv, 14, 15, 16, 36, 37, 40, 64
Workbench 34, 36, 67